

TECHNICAL DOCUMENTATION

SGLottery User Manual

Singapore 4D & TOTO Ticket Scanner Application

VERSION

2.0

DATE

March 2026

PLATFORM

Web · Android · iOS

React + Vite

React Native + Expo

Express.js

Firebase Firestore

Tesseract.js + OCR.Space

Puppeteer

Table of Contents

1. Overview

[Platform Support](#)

2. Project Orientation — Read This First

[What This App Does \(Plain English\)](#)

[The Four Components](#)

[How They Talk to Each Other](#)

[Design Pattern: VIPER \(Mobile\) / Presenter \(Web\)](#)

[Design System \(Colours\)](#)

[Git Branches](#)

3. Prerequisites & Dependencies

[External Services Checklist](#)

4. Installation & Setup

[Quick Setup Status Checklist](#)

5. Environment Configuration

[Backend `.env` reference](#)

[Firestore collections \(auto-created on first backend start\)](#)

6. Running the Applications

[Documentation \(rebuild HTML + PDF\)](#)

7. Codebase Walkthrough by Web-Flow

[7.1 User Opens the App \(Web\)](#)

[7.2 User Uploads a Ticket](#)

[7.3 Backend: OCR + Parsing](#)

[7.4 Backend: Result Checking + Notifications](#)

[7.5 Results Scraper](#)

[7.6 Prize Matching Logic](#)

[7.7 History Page](#)

[7.8 Mobile App](#)

8. Feature Guide

[Uploading a Ticket](#)

[Viewing Draw Results](#)

[Ticket History](#)

[Push Notifications \(Web\)](#)

[Push Notifications \(Mobile\)](#)

9. Predictive Analysis Guide

[Model 1: Digit Frequency Analysis \(`modelId: "frequency"`\)](#)

[Model 2: Hot & Cold Number Analysis \(`modelId: "hot\\_cold"`\)](#)

[Model 3: Odd & Even / Jackpot Pattern Analysis \(`modelId: "odd\\_even"`\)](#)

[How to Use the Predict Page](#)

10. Testing Each Feature

[Manual Testing Checklist](#)

11. Troubleshooting

[Backend will not start](#)

[OCR returns no numbers](#)

[OCR Known Fault Case — Partial Number Extraction](#)

[Settings tab crashes on Android](#)

["Unmatched Route" error on development build](#)

[Tab bar hidden behind phone navigation buttons](#)

[Push notifications not arriving \(mobile\)](#)

[Singapore Pools scraping fails](#)

12. Known Issues

13. Data Privacy & Security

[What Data is Stored](#)

[What is NOT Stored](#)

[Security Practices](#)

14. Architecture Overview

[Data Flow for a New Ticket Upload](#)

15. API Reference

[Base URL](#)

[Authentication](#)

[POST `/api/tickets/upload`](#)

[GET `/api/tickets`](#)

[GET `/api/tickets/:id`](#)

[DELETE `/api/tickets/:id`](#)

[GET `/api/results/4d`](#)

[GET `/api/results/toto`](#)

[POST `/api/results/check/:ticketId`](#)

[GET `/api/predict`](#)

[POST `/api/devices/register`](#)

16. How to Make a Change (Intern Development Loop)

[Common mistakes interns make — avoid these](#)

17. Intern Challenge — Improve OCR Accuracy

[Background: The Problem](#)

[Why It Happens — Technical Detail](#)

[Three Approaches to Investigate](#)

[How to Measure Success](#)

[Submitting Your Fix](#)

SGLottery — User Manual

Singapore 4D & TOTO Ticket Scanner Application

Version 2.0 | March 2026

1. Overview

SGLottery is a full-stack application for scanning, storing, and analysing Singapore Pools 4D and TOTO lottery tickets. It supports:

- **Ticket photo upload** with OCR number extraction
- **TOTO System Bet expansion** (all $C(n,6)$ combinations)
- **Automated result checking** against Singapore Pools
- **Push notifications** (win/loss alerts on web and mobile)
- **Predictive analysis** using 3 statistical models
- **Past results browser** (4D and TOTO)
- **Full ticket history** with sorting, filtering, and detail view

Platform Support

Platform	Status
Web (Desktop)	✓ Full support
Mobile Web (browser)	✓ Responsive
Android (Expo)	✓ Full support
iOS (Expo)	✓ Full support

2. Project Orientation — Read This First

If you are a new intern or developer joining this project, read this section before touching any code.

What This App Does (Plain English)

A user photographs their Singapore Pools lottery ticket with their phone or computer. The app extracts the numbers using OCR (Optical Character Recognition), saves them to a database, then automatically checks Singapore Pools after the draw date to see if the ticket won. The user gets a push notification with the result.

The Four Components

There are four separate programs that make up this app. They each run as their own server/process:

```
4d-toto-app/
├── backend/      ← Express.js server (Node.js) – the brain
                  Handles OCR, database, Singapore Pools scraping
├── web-app/      ← React + Vite website (runs in browser)
                  For desktop users: upload, history, results, predict
├── mobile-app/   ← React Native + Expo (runs on phone)
                  Same features as web-app but as a mobile app
└── docs/        ← Documentation (you are here)
                  USER_MANUAL.md – source (edit this)
                  USER_MANUAL.html / .pdf – generated (never edit directly)
                  build.js – run this to regenerate HTML + PDF
```

How They Talk to Each Other

```
Web or Mobile App
|
| HTTP requests (fetch / axios)
▼
Backend API (http://localhost:3001)
|
├── Firebase Firestore (database, stores tickets + results)
├── OCR.Space API      (primary OCR engine, cloud)
├── Tesseract.js       (fallback OCR engine, local)
└── Singapore Pools   (scrapes live results via Puppeteer)
```

Design Pattern: VIPER (Mobile) / Presenter (Web)

Both the web and mobile apps use a **Presenter** pattern:

- **Presenter** = fetches data from the backend, stores it in React state, passes it down via Context
- **View** = dumb component that receives data as props and renders it

- **Page/Screen** = thin shell that imports the View

This means if you want to change what data is loaded, edit the Presenter. If you want to change how it looks, edit the View. They are kept separate on purpose.

Layer rules — commit these to memory:

This project actually uses a **4-layer architecture** — there is an additional **Interactor** layer between the Presenter and the API:

```
Page / Screen
└─ View      (renders UI, receives data as props)
    └─ Presenter (holds React Context + state, triggers the Interactor)
        └─ Interactor (useTicketInteractor / useResultsInteractor – calls the API)
            └─ api.ts / api.js (central fetch wrapper – the only file that calls the API)
```

Layer	File location	Job	What it must NOT do
Page / Screen	pages/ or app/tabs/	Import and render the View	Any logic at all — thin shell only
View	views/	Render UI from props/Context	Call <code>fetch()</code> , hold state, business logic
Presenter	presenters/	Provide data via Context, trigger Interactor on mount	Contain JSX styling
Interactor	interactors/	<code>useState</code> + <code>useCallback</code> , calls <code>api.get/post</code>	Render any JSX
API service	services/api.ts or services/api.js	Central <code>fetch</code> wrapper	Business logic

Wrong vs. right — the most common intern mistake:


```

// WRONG – fetch() inside a View component
// The View now "knows" where its data comes from. Breaks the separation.
export function HistoryView() {
  const [tickets, setTickets] = useState([]);
  useEffect(() => {
    fetch('/api/tickets').then(r => r.json()).then(setTickets); // ← NO
  }, []);
  return <FlatList data={tickets} />;
}

// CORRECT – each layer does its one job:

// 1. Interactor (interactors/useTicketInteractor.js) – owns the fetch
export function useTicketInteractor() {
  const [tickets, setTickets] = useState([]);
  const fetchTickets = useCallback(async () => {
    const data = await api.get('/tickets'); // ← api.get is the only place to
    setTickets(data.tickets ?? []);
  }, []);
  return { tickets, fetchTickets };
}

// 2. Presenter (presenters/TicketPresenter.jsx) – calls Interactor, shares via Context
export function TicketPresenter({ children }) {
  const ix = useTicketInteractor();
  useEffect(() => { ix.fetchTickets(); }, []); // triggers on mount
  return <Ctx.Provider value={{ tickets: ix.tickets }}>{children}</Ctx.Provider>;
}

// 3. View (views/history/HistoryView.jsx) – reads Context, renders only
export function HistoryView() {
  const { tickets } = useTickets(); // ← reads from Context, never fetches
  return <FlatList data={tickets} />;
}

```

Key files:

- `web-app/src/presenters/TicketPresenter.jsx` — loads ticket list, provides it to all pages
- `mobile-app/src/presenters/TicketPresenter.tsx` — same for mobile
- `mobile-app/src/presenters/ResultsPresenter.tsx` — loads results data for mobile

Design System (Colours)

Both web and mobile use the same colour tokens. If you change a colour, change it in both places:

Token	Hex	Used for
Background	#0d1117	Page backgrounds
Surface	#161b22	Cards, panels
Border	#21262d	Dividers, outlines
4D Accent	#e53935	Singapore red, 4D badges
TOTO Accent	#7c3aed	Purple, TOTO badges
Gold	#f59e0b	Winners, highlights
Cyan	#22d3ee	Links, info states
Text	#f0f6fc	Primary text
Muted	#8b949e	Labels, secondary text

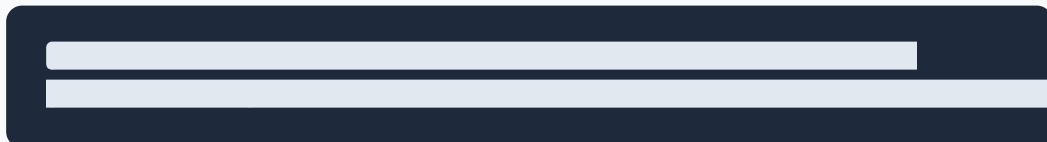
Web: defined in `web-app/src/index.css` as CSS variables (`--bg` , `--surface` , etc.) Mobile: defined as a `C = { ... }` object in each screen/view file using `StyleSheet.create()`

Git Branches

- `main` — stable, production-ready code. Do not push broken code here.
- `develop` — your working branch. Merge your changes here first, then PR to main.

Always create a new branch off `develop` for your feature: `git checkout -b feature/my-feature develop`

Note for new interns: The `develop` branch may not exist yet in a fresh clone. If `git checkout develop` gives an error, create it first:



From that point, all feature branches go off `develop` , not `main` . Never push directly to `main` .

3. Prerequisites & Dependencies

Before you touch any code, run these checks. Each tool has a **verify command** — if you see the expected output, you are good. If you see an error or wrong version, follow the fix steps.

Tool 1 — Node.js (minimum v18)

Check if already installed:

```
node -v
```

Expected output (good):

```
v20.11.0
```

Any `v18.x.x` or higher is fine. If you see `v16.x.x` or lower, upgrade.

Not installed or wrong version — fix:

1. Go to nodejs.org (<https://nodejs.org>) and download the **LTS** version
2. Run the installer — it also installs `npm` automatically
3. Re-run `node -v` to confirm

Reset a broken Node.js installation:

- Windows: Uninstall Node.js from Control Panel → Programs, then reinstall from nodejs.org
 - If you have `nvm` installed: `nvm install 20 && nvm use 20`
-

Tool 2 — npm (included with Node.js)

Check if already installed:

```
npm -v
```

Expected output (good):

```
10.2.4
```

Any `v9.x.x` or higher is fine.

Wrong version — fix:

```
npm install -g npm@latest
```

Tool 3 — Git

Check if already installed:

```
git --version
```

Expected output (good):

```
git version 2.43.0.windows.1
```

Not installed — fix: Download from git-scm.com (<https://git-scm.com>) and run the installer.

Tool 4 — Expo CLI

Check if already installed:

```
npx expo --version
```

Expected output (good):

```
0.18.28
```

Not installed — fix:

```
npm install -g expo-cli
```

Tool 5 — EAS CLI (only needed if building a development APK)

Check if already installed:

```
eas --version
```

Expected output (good):

```
eas-cli/9.0.0 win32-x64 node-v20.11.0
```

Not installed — fix:

```
npm install -g eas-cli
```

External Services Checklist

Before proceeding, confirm you have access to these:

Service	How to check you have access	Free?
Firebase project	Ask the team for the <code>serviceAccountKey.json</code> file	Yes (Spark plan)
Firebase Firestore	Firestore must be enabled in the Firebase console	Yes
OCR.Space	App uses public <code>helloworld</code> key by default — no signup needed	Yes
ngrok	Download from ngrok.com (https://ngrok.com).	Yes (1 tunnel)
Expo account	<code>npx expo whoami</code> — shows your username if logged in	Yes

4. Installation & Setup

Work through each step in order. Do not skip a step even if it seems already done.

Step 1 — Clone the repository

```
git clone https://github.com/ValentineVirgo123/4d-toto-app.git
cd 4d-toto-app
```

Verify it worked:

```
ls
```

Expected output:

```
backend/  mobile-app/  web-app/  docs/  README.md
```

If you see these 4 folders, the clone was successful.

Step 2 — Install dependencies for each component

Run these one by one. Each takes 1–3 minutes.

Backend:

```
cd backend
npm install
```

Expected output (last line):

```
added 312 packages, and audited 313 packages in 45s
```

Web app:

```
cd ../web-app
npm install
```

Expected output (last line):

```
added 284 packages, and audited 285 packages in 30s
```

Mobile app:

```
cd ../mobile-app
npm install
```

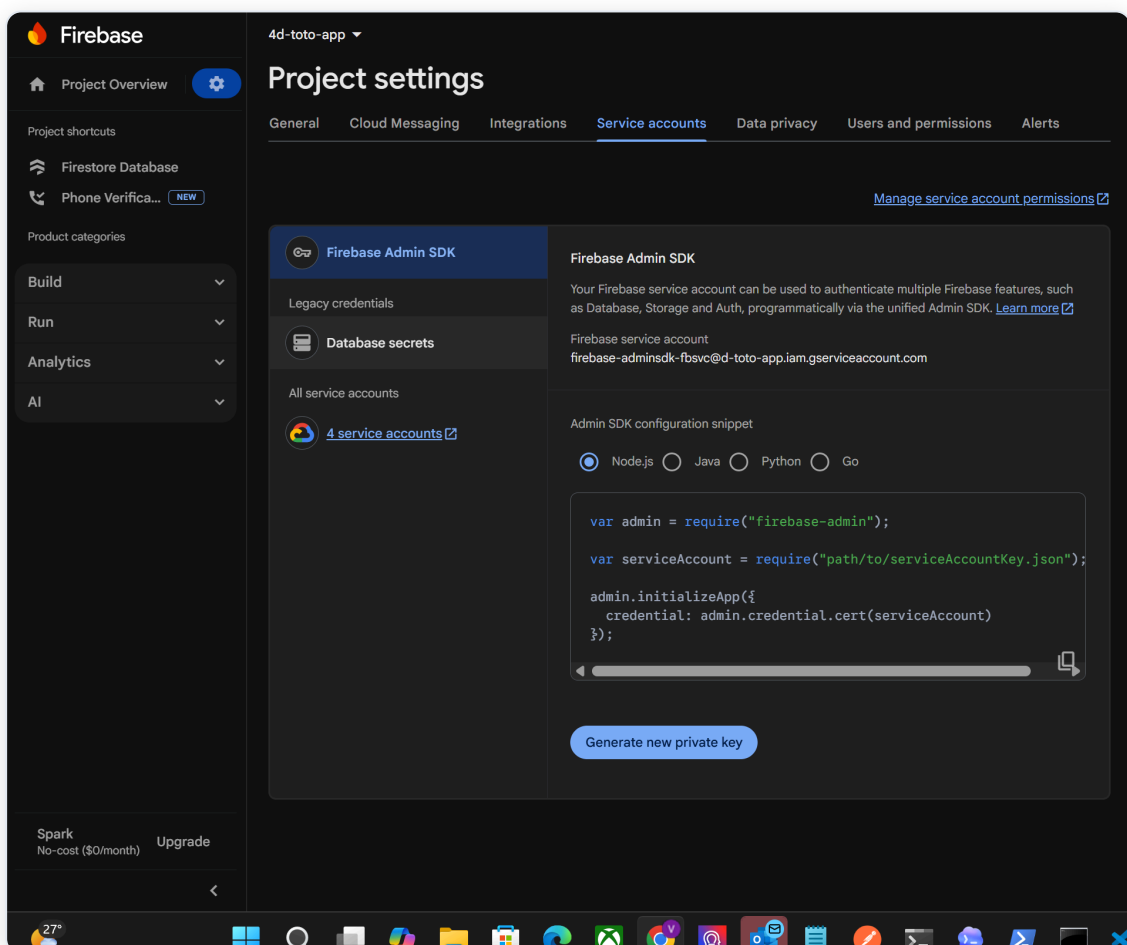
Expected output (last line):

```
added 1247 packages, and audited 1248 packages in 2m
```

If you see errors during npm install: The most common cause is an old or corrupted `node_modules` folder from a previous setup. Reset it:

```
rm -rf node_modules
npm install
```

Step 3 — Set up Firebase



Firebase Console

1. Go to console.firebase.google.com (<https://console.firebase.google.com>),
2. Open the project (ask the team which project is being used)
3. Click **Project Settings** (gear icon, top left)
4. Click the **Service Accounts** tab
5. Click **Generate new private key** → **Generate Key**
6. A JSON file downloads — rename it `serviceAccountKey.json`
7. Move it into the `backend/` folder

Verify it is in the right place:

```
ls backend/serviceAccountKey.json
```

Expected output:

```
backend/serviceAccountKey.json
```

Never commit this file. It is already in `.gitignore`. If you accidentally committed it, rotate the key immediately in Firebase console.

Step 4 — Configure environment variables

```
cd backend
cp .env.example .env
```

Open `backend/.env` and fill in the three Firebase values. You get these by opening your `serviceAccountKey.json` :

```
FIREBASE_PROJECT_ID=your-project-id          ← "project_id" in the JSON
FIREBASE_CLIENT_EMAIL=abc@project.iam.gserviceaccount.com ← "client_email" in the
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
PORT=3001
```

Verify the file exists and is not empty:

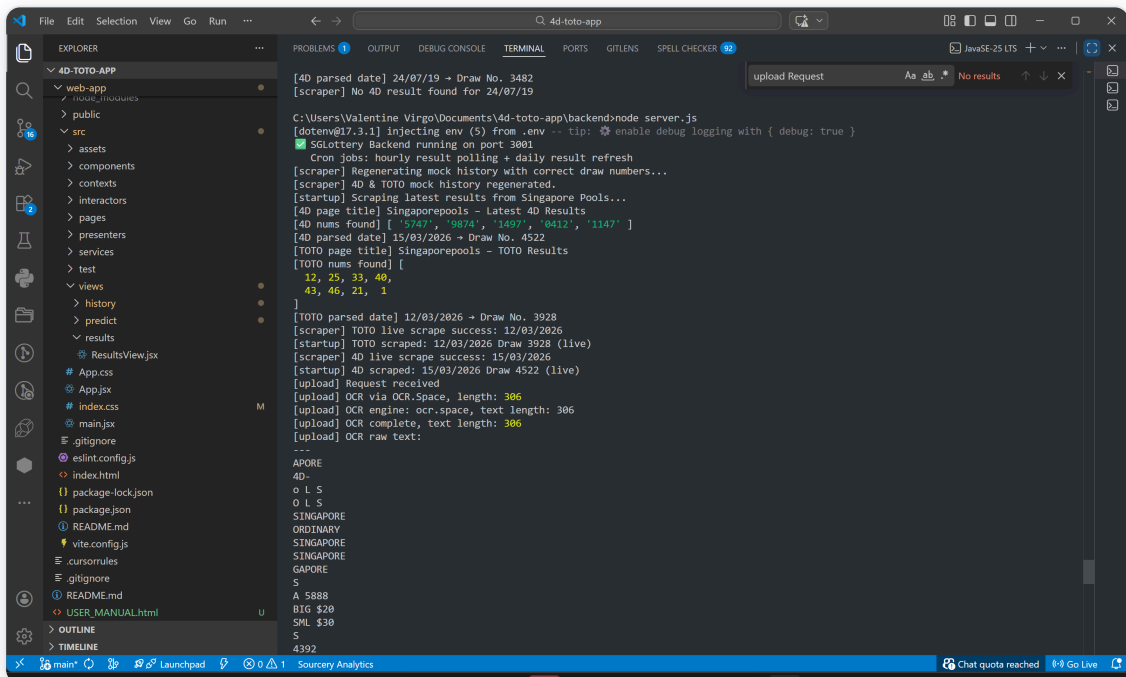
```
cat backend/.env
```


Expected output (values will be yours):

```
FIREBASE_PROJECT_ID=sglottery-xxxxx
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxx@sglottery-xxxxx.iam.gserviceaccount.com
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nMIIEv..."
PORT=3001
```

Common mistake: Copying the `FIREBASE_PRIVATE_KEY` without quotes, or with literal newlines instead of `\n`. The entire key must be on one line, wrapped in double quotes.

Step 5 — Verify the backend starts correctly



Backend terminal

```
cd backend
node server.js
```

Expected output (within 10 seconds):

```
✅ SGLottery Backend running on port 3001
  Cron jobs: hourly result polling + daily result refresh
[scraper] Regenerating mock history with correct draw numbers...
[scraper] 4D & TOTO mock history regenerated.
[startup] Scraping latest results from Singapore Pools...
[startup] 4D scraped: 15/03/2026 Draw 4522 (live)
[startup] TOTO scraped: 13/03/2026 Draw 3927 (live)
```

If the Singapore Pools scrape fails (SP website blocked), you will see `mock` instead of `live` — this is normal and the app still works with mock data.

If you see a Firebase credentials error:

```
Error: Could not load the default credentials
```

→ Your `FIREBASE_PRIVATE_KEY` in `.env` is malformed. Make sure it is wrapped in double quotes and contains `\n` (not actual newlines).

If you see port already in use:

```
Error: listen EADDRINUSE: address already in use :::3001
```

→ Kill the old process first:

```
# Windows
netstat -ano | findstr :3001
taskkill /PID <the-PID-number> /F

# Mac/Linux
lsof -ti:3001 | xargs kill
```

Step 6 — Create a test account

The app requires a login. You need to create a Firebase Auth account before you can use the app.

Option A — via the web app sign-up screen (easiest):

1. Start the backend: `cd backend && node server.js`
2. Start the web app: `cd web-app && npm run dev`

3. Open `http://localhost:5173` — you will see the login/home page
4. Click **Sign Up** (or **Get Started**) and register with any email + password
5. Use these same credentials every time you log in

Tip: Use a throwaway email like `test@test.com` with password `test1234` for local development. This is only stored in your Firebase project — it is not a real account anywhere.

Option B — via Firebase Console (if sign-up is disabled in the app):

1. Go to console.firebase.google.com (<https://console.firebase.google.com>) → your project
2. Click **Authentication** → **Users** tab → **Add user**
3. Enter any email + password and click **Add user**
4. Use those credentials to log in to the web or mobile app

Verify you are logged in: After signing in, the sidebar with Upload / History / Results / Predict / Settings should appear.

Step 7 — Verify the web app starts

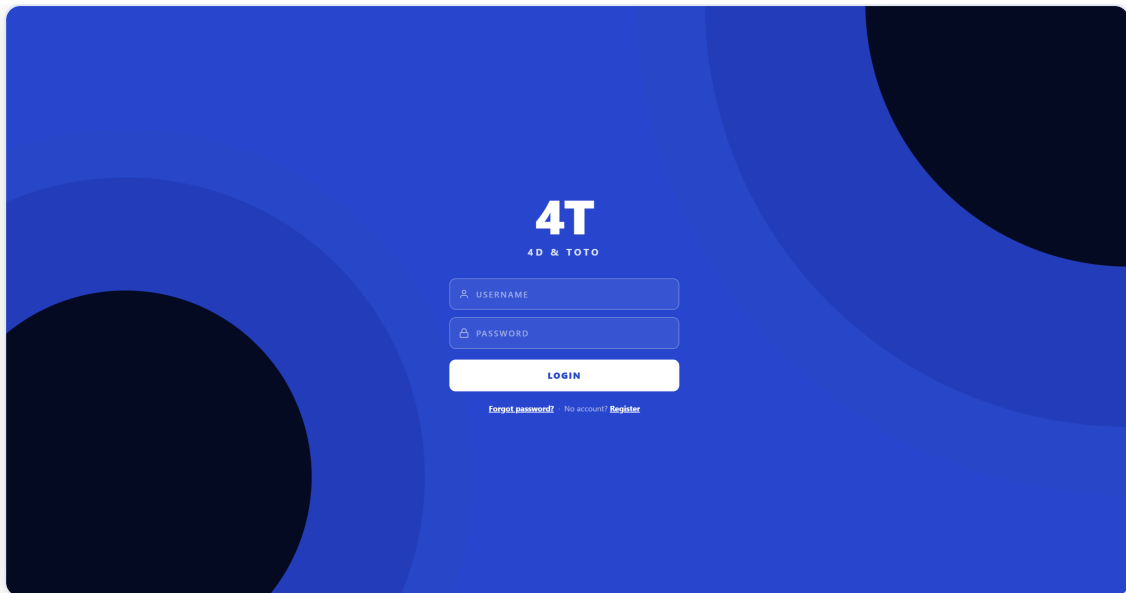
```
cd web-app
npm run dev
```

Expected output:

```
VITE v5.x.x ready in 800ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
```

Open `http://localhost:5173` in your browser. You should see the SGLottery login/home page.



Login page

If the page loads but shows no data / blank: Make sure the backend is running first (`node server.js` in a separate terminal).

Step 8 — Verify the mobile app starts (Expo Go)

```
cd mobile-app
npx expo start
```

Expected output:

```
Starting project at /path/to/mobile-app
...
> Metro waiting on exp://192.168.x.x:8081
> Scan the QR code above with Expo Go (Android) or Camera app (iOS)
```

```
C:\Users\Valentine Virgo\Documents\4d-toto-app\mobile-app>npx expo start
Starting project at C:\Users\Valentine Virgo\Documents\4d-toto-app\mobile-app
React Compiler enabled
Starting Metro Bundler
```



```
> Metro waiting on exp://192.168.0.103:8081
> Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

> Web is waiting on http://localhost:8081

> Using Expo Go
```

Expo QR code

Open the **Expo Go** app on your phone, tap **Scan QR Code**, and point at the terminal QR code. The app should load within 30 seconds.

Phone cannot connect? Make sure your phone and computer are on the **same WiFi network**. If they are on different networks, the QR code URL won't be reachable.

Limitation: Expo Go does not support real push notifications. For full push notification support, use the development APK build instead (`npx expo start --dev-client`).

Step 9 — Connect mobile app to backend (ngrok or same WiFi)

The mobile app needs to reach the backend. There are two options:

Option A — Same WiFi (simpler, recommended for local dev):

1. Find your computer's local IP address:

```
# Windows
ipconfig
# look for "IPv4 Address" under your WiFi adapter, e.g. 192.168.1.105

# Mac/Linux
ifconfig | grep "inet "
```

2. Create your local env file from the template, then set the URL:

```
cp mobile-app/.env.example mobile-app/.env
```

Open `mobile-app/.env` and set:

```
EXPO_PUBLIC_API_URL=http://192.168.1.105:3001/api
```

Why `.env` and not the source file? The API URL is read from `process.env.EXPO_PUBLIC_API_URL` in `mobile-app/src/services/api.ts`. Environment variables keep your local IP out of version control — everyone on the team has a different IP. `mobile-app/.env` is git-ignored; `mobile-app/.env.example` is the safe committed template.

3. Restart Expo: `npx expo start`

Option B — ngrok (works across networks):

```
ngrok
ngrok
(CTRL+C to quit)
(CTRL+C to quit)

💡 One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             valentinevirgo443@gmail.com (Plan: Free)
Update              update available (version 3.37.2, Ctrl-U to update)
Version             3.37.1
Region              Asia Pacific (ap)
Latency             3ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://nonvascular-martina-nonfacetiously.ngrok-free.dev -> http://localhost:3001

Connections         ttl    opn    rt1    rt5    p50    p90
                   0      0      0.00   0.00   0.00   0.00
```

ngrok tunnel

1. Open a new terminal (keep backend running in the first one)

2. Run:

```
ngrok http 3001
```

3. Copy the `https://xxxx.ngrok-free.app` URL from the ngrok output

4. Update `mobile-app/.env` (create from `.env.example` first if needed):

```
EXPO_PUBLIC_API_URL=https://xxxx.ngrok-free.app/api
```

The `ngrok-skip-browser-warning: true` header is already included automatically in every request by `mobile-app/src/services/api.ts` — you do not need to add it manually.

5. Restart Expo: `npx expo start`

Verify the connection works: Upload a ticket in the mobile app. If the OCR result appears, the connection is working.

ngrok free plan allows only 1 tunnel. If you already have a ngrok tunnel running for something else, you cannot run a second one. Use Option A (same WiFi) instead.

Quick Setup Status Checklist

Run through this before asking for help:

- ☐ `node -v` → v18.x.x or higher
- ☐ `npm -v` → v9.x.x or higher
- ☐ `git --version` → git version 2.x.x
- ☐ `backend/serviceAccountKey.json` exists
- ☐ `backend/.env` exists with all 3 Firebase values filled in
- ☐ `cd backend && node server.js` → "✅ SGLottery Backend running on port 3001"
- ☐ `cd web-app && npm run dev` → "Local: http://localhost:5173"
- ☐ `cd mobile-app && npx expo start` → QR code appears in terminal
- ☐ Mobile app loads on phone → Home screen visible
- ☐ Upload a test ticket → OCR result displayed

If any box is not checked, re-do that step above.

5. Environment Configuration

Backend `.env` reference

```
# — Firebase Admin SDK (required) —
FIREBASE_PROJECT_ID=your-project-id
FIREBASE_CLIENT_EMAIL=your-client@project.iam.gserviceaccount.com
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nMIIEv...\n-----END PRIVATE KEY-----"

# — Server —
PORT=3001

# — Optional: Google Vision API (not used by default – OCR.Space is primary) —
GOOGLE_VISION_KEY_PATH=./google-vision-key.json
```

Firestore collections (auto-created on first backend start)

Collection	Contents
<code>tickets</code>	Scanned lottery tickets
<code>results_4d</code>	4D draw results (live or mock)
<code>results_toto</code>	TOTO draw results (live or mock)
<code>devices</code>	Mobile push notification tokens
<code>users</code>	User profiles (if auth is enabled)

For interns coming from SQL / relational databases — read this:

Firestore is a **NoSQL document database**. It works very differently from MySQL or PostgreSQL:

Concept	SQL (MySQL)	Firestore (NoSQL)
Data container	Table (fixed schema)	Collection (no schema)
Row	Row (all same columns)	Document (each can have different fields)
Lookup	<pre>SELECT * FROM tickets WHERE id = 1</pre>	<pre>db.collection('tickets').doc('abc123').get()</pre>
Filter	<pre>WHERE gameType = '4D'</pre>	<pre>.where('gameType', '==', '4D')</pre>
Join	<pre>JOIN results ON ...</pre>	No joins. Make two separate queries.
ID	Auto-increment integer	Auto-generated string (e.g. <code>'XkP2mNqLr9vTa8'</code>)

Key implication: Because there are no joins, if you need both a ticket and its draw result at the same time, you must make two separate `.get()` calls and combine them in JavaScript. This is intentional — Firestore prioritises read speed at massive scale over query flexibility.

No schema enforcement: If you save `{ gameType: 'TOTO', numbers: [3,12,18] }` to one document and `{ gameType: '4D', drawDate: '15/03/2026' }` to another, Firestore will accept both. It does not complain about missing fields. This means your JavaScript code must handle missing fields gracefully (`ticket.numbers ?? []`).

6. Running the Applications

Always start in this order: **Backend first → Web or Mobile second.**

Backend

```
cd backend
node server.js
```

Success:

✅ SGLottery Backend running on port 3001

Web App

```
cd web-app
npm run dev
```

Success: Open <http://localhost:5173>

Mobile App (Expo Go — for most development)

```
cd mobile-app
npx expo start
```

Success: QR code appears. Scan with Expo Go app on phone.

Mobile App (Development Build — for push notification testing)

```
cd mobile-app
npx expo start --dev-client
```

Requires the development APK installed on the phone. The phone appears as a selectable device in the terminal.

How to build and install the development APK (one-time setup):

1. Log in to Expo:

```
npx expo login
```

2. Build the development APK (takes 5–15 minutes on Expo's cloud servers):

```
cd mobile-app
eas build --profile development --platform android
```

3. When the build finishes, Expo prints a download URL. Download the `.apk` file.
4. Transfer the `.apk` to your Android phone (via USB, Google Drive, or email) and install it.
 - On the phone: **Settings** → **Security** → **Allow installs from unknown sources** (exact path varies by Android version)
5. Open the installed app — it shows a QR code scanner. Scan the QR code from `npx expo start --dev-client`.

iOS: Development builds on iOS require an Apple Developer account (\$99/year). For iOS testing without a paid account, use Expo Go (push notifications will not work).

Documentation (rebuild HTML + PDF)

Whenever `docs/USER_MANUAL.md` is updated, regenerate the HTML and PDF so they stay in sync. Always edit the `.md` file — never edit the HTML or PDF directly.

```
# From the project root (4d-toto-app/)

node docs/build.js          # rebuilds both HTML and PDF (~30 sec)
node docs/build.js --html    # HTML only, faster – use this to preview changes
```

Output file	What it's for
<code>docs/USER_MANUAL.html</code>	Share or open in browser — fully self-contained, screenshots embedded
<code>docs/USER_MANUAL.pdf</code>	Print or distribute — TOC entries are clickable and jump to the right page

Note: `USER_MANUAL.html` and `USER_MANUAL.pdf` are in `.gitignore` (too large for git). Always regenerate them locally after pulling new changes to the `.md` file.

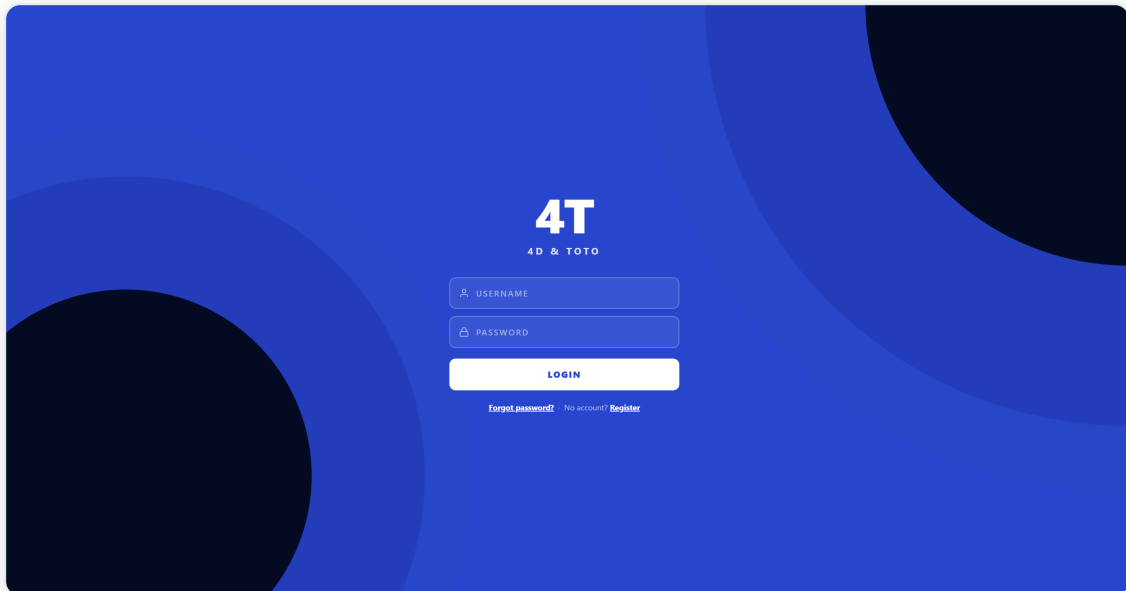
7. Codebase Walkthrough by Web-Flow

This section follows the exact path a user takes through the app, and explains each file involved.

7.1 User Opens the App (Web)

File: `web-app/src/App.jsx`

```
User opens browser → App.jsx loads → checks if user is logged in
├─ Not logged in → shows HomePage (landing page / login)
└─ Logged in    → shows Sidebar + pages (Upload, History, Results, Predict, Set)
```



Login / Home Page

🔑 Heart of the Logic — `web-app/src/App.jsx` · Lines 28 – 49

What it does: Controls which UI the user sees based on login state. If `user` is null (not logged in), only the landing/login page is shown. Once logged in, the full app with sidebar and all routes is rendered.

How to navigate to this code:

1. Open `web-app/src/App.jsx`
2. Press `Ctrl+G` → type `28` → Enter — OR — press `Ctrl+F` and search for `if (!user)`

```
// — Lines 28-34 — NOT logged in: show login page only —————
if (!user) {
  return (
    <Routes>
      <Route path="/" element={<HomePage />} />
    </Routes>
  );
}

// — Lines 43-49 — Logged in: show all app routes inside the sidebar shell —
<Route path="/" element={<Navigate to="/upload" replace />} />
<Route path="/upload" element={<UploadPage />} />
<Route path="/history" element={<HistoryPage />} />
<Route path="/results" element={<ResultsPage />} />
<Route path="/predict" element={<PredictPage />} />
<Route path="/settings" element={<SettingsPage />} />
```

Key concept: `AuthProvider` (wrapped around everything in `App.jsx`) holds the login state. `useAuth()` reads it. The entire app routing depends on whether `user` is null or not.

🔑 Secondary Logic — `web-app/src/App.jsx` · Lines 21 – 24

What it does: Polls for pending draw notifications every 60 seconds. This is the web fallback for push notifications (browsers cannot receive Expo push).

How to navigate:

1. Open `web-app/src/App.jsx`
2. Press `Ctrl+G` → type `21` → Enter — OR — search for `checkPendingNotifications`

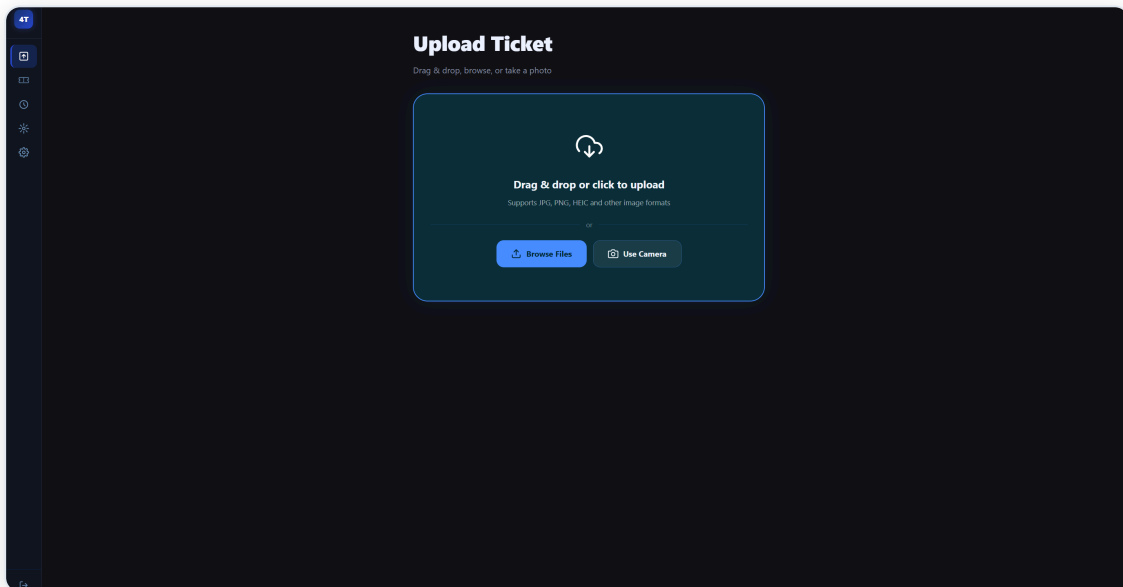
```
// — Lines 21-24 —————
useEffect(() => {
  checkPendingNotifications(); // run once on mount
  const interval = setInterval(checkPendingNotifications, 60_000); // then every 60s
  return () => clearInterval(interval);
}, []);
```

7.2 User Uploads a Ticket

File: `web-app/src/pages/UploadPage.jsx`

This is the most important user-facing page. The full flow when a user drops an image:

```
User drops image
  → processFile() is called
  → FormData with the image is sent to POST /api/tickets/upload
  → Backend responds with parsed ticket data
  → Result is shown: numbers, draw date, win/loss status
  → If future draw: browser notification is scheduled
```



Upload Page

🔑 Heart of the Logic — `web-app/src/pages/UploadPage.jsx` · Lines 338 – 376 · `processFile()`

What it does: The entire upload flow lives in this one async function. It validates the file, shows a preview, sends to the backend via FormData, stores the response, and schedules a browser notification if the draw is future. Everything the user sees after dropping an image flows through here.

How to navigate to this code:

1. Open `web-app/src/pages/UploadPage.jsx`
2. Press `Ctrl+G` → type `338` → Enter — OR — press `Ctrl+F` and search for `const processFile`

```
const processFile = useCallback(async (file) => {
  // Step 1: Validate it's an image
  if (!file || !file.type.startsWith('image/')) {
    setError('Please upload an image file (JPG, PNG, HEIC, etc.).');
    return;
  }

  // Step 2: Show a preview immediately (before upload completes)
  setPreview(URL.createObjectURL(file));
  setLoading(true);

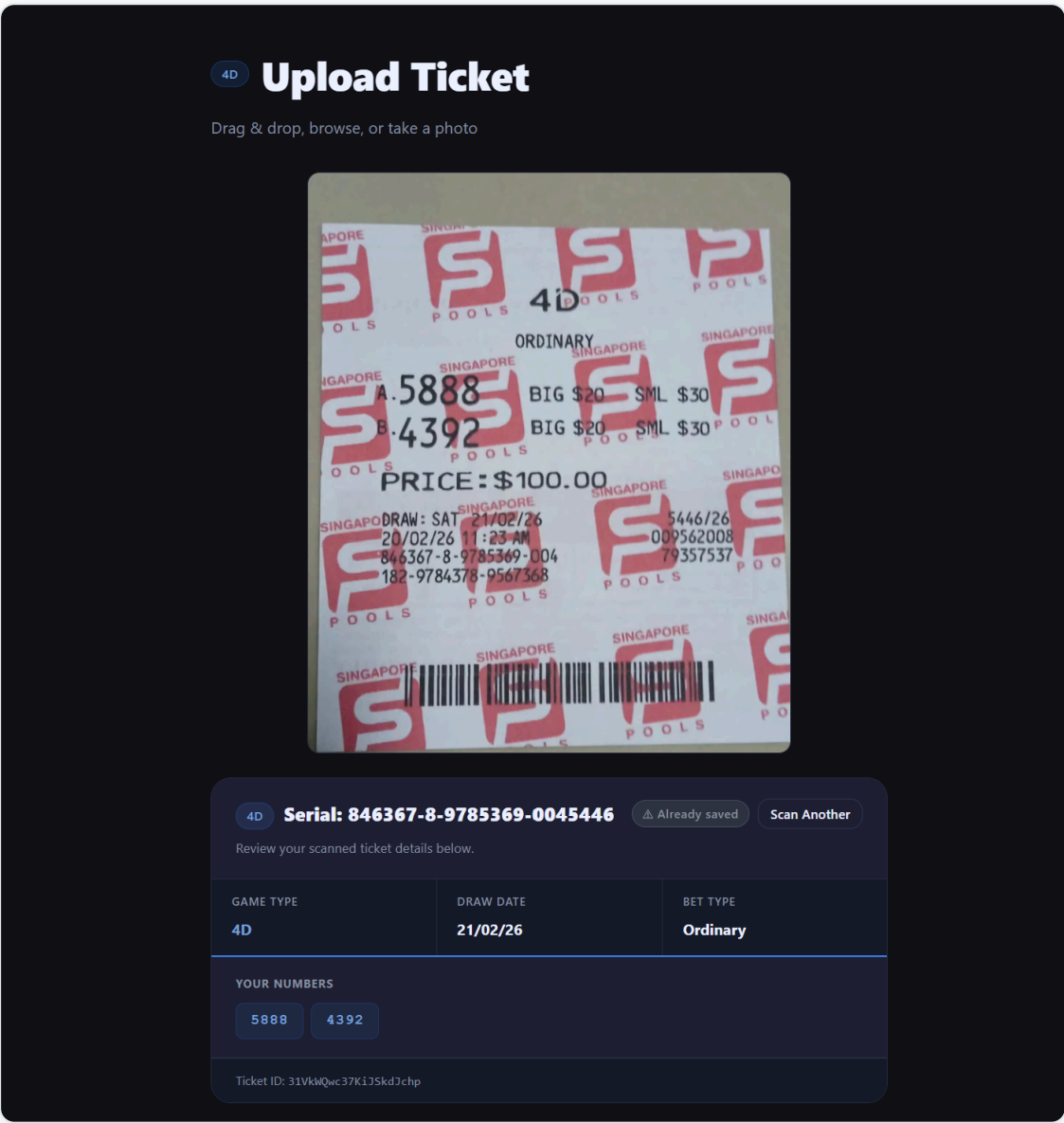
  // Step 3: Package as multipart/form-data and send to backend
  const form = new FormData();
  form.append('ticket', file);
  const res = await fetch(`${BASE_URL}/tickets/upload`, { method: 'POST', body: form });
  const data = await res.json();

  // Step 4: Store result in state → triggers re-render
  setResult(data);

  // Step 5: If the draw hasn't happened yet, schedule a browser notification
  if (data.drawType === 'future' && data.id) {
    scheduleDrawNotification({ id: data.id, gameType: data.gameType, drawDate: data.drawDate });
  }
}, []);
```

What the result card shows:

<code>result.drawType</code>	What the user sees
<code>'past'</code> + won	Green banner, prize tier, winnings calculation
<code>'past'</code> + not won	Red banner, official draw numbers shown
<code>'future'</code>	Blue banner, timeline, "Enable Notifications" button



Successful OCR scan — 5888 and 4392 correctly extracted, draw date 21/02/26, Ordinary bet

Above: a clean successful scan. The app correctly identified both numbers (A. 5888, B. 4392), the draw date, and bet type from a heavily watermarked Singapore Pools ticket.

🔑 Secondary Logic — `web-app/src/pages/UploadPage.jsx` · Lines 8 – 22 · Prize rate table

What it does: `calc4DWinnings()` uses a hardcoded Singapore Pools prize table to instantly calculate winnings: e.g. a \$1 Big bet on 1st Prize = \$2,000 payout. No API needed — the rates are fixed by Singapore Pools rules.

How to navigate:

1. Open `web-app/src/pages/UploadPage.jsx`
 2. Press `Ctrl+G` → type `8` → Enter — OR — search for `PRIZE_RATES_4D`
-

7.3 Backend: OCR + Parsing

File: `backend/routes/tickets.js`

This is the most complex file in the entire project. It handles the full ticket processing pipeline.

```
POST /api/tickets/upload
↓
Image received (multer)      ← multer is an Express middleware that parses
                              multipart/form-data requests – the encoding
                              browsers use when sending files. Without it,
                              req.file would be undefined and the image
                              would never reach the backend.
                              ↓
OCR.Space API called (primary) → if fails → Tesseract.js (fallback)
                              ↓
parseTicketText() called on raw OCR text
                              ↓
Result: { gameType, drawDate, numbers, betType, systemSize, ... }
                              ↓
If TOTO System Bet → expandSystemBet() → expandedCombinations array
                              ↓
classifyDrawDate() → 'past' or 'future'
                              ↓
If past → scrape result → compare → add win/loss data
                              ↓
Save to Firestore tickets collection
                              ↓
Return full JSON response to frontend
```

4D Upload Ticket

Drag & drop, browse, or take a photo



4D Serial: 846367-8-9785369-0045446

Already saved

Scan Another

Review your scanned ticket details below.

GAME TYPE	DRAW DATE	BET TYPE
4D	21/02/26	Ordinary
YOUR NUMBERS		
5888 4392		
Ticket ID: 31VklQwc37K135kdJchp		

OCR Result — Successful extraction

What a successful OCR result looks like: The user photographs their ticket → the backend extracts every number correctly → the result card appears with:

- **All numbers** from the ticket shown in the numbered list
- **Draw date** matched and classified as past or future
- **Game type** (4D red / TOTO purple) detected automatically
- If the draw has passed: a **win/loss verdict** with prize tier and winnings calculation
- If the draw is upcoming: a **blue "Future Draw" banner** with a notification button

This is the happy path. When OCR.Space is the engine used, this result is reliable. A known edge case where numbers can be silently dropped is documented in [Section 11 — OCR Known Fault Case](#).

OCR ENGINE OVERVIEW — WHY THREE ENGINES, AND WHAT'S BROKEN

The backend tries three OCR engines in priority order. Understanding each one is essential before attempting to improve accuracy.

Engine	Status	Where	Strength	Weakness
OCR.Space	✓ Active (primary)	Cloud API	Handles watermarks well; higher accuracy	Rate-limited on <code>helloworld</code> key (500 req/month); needs paid key for production
Tesseract.js	✓ Active (fallback)	Runs locally on server	No internet needed; free	Struggles with the Singapore Pools watermark even after removal; can silently drop numbers
Google Cloud Vision	✗ Not wired up	Cloud API	Best accuracy; purpose-built for OCR	Requires a billing account — the project has a Google Cloud account set up but cannot activate billing; no code uses it yet

Why Google Vision is not working (`backend/.env` → `GOOGLE_VISION_KEY_PATH`): The `.env` file has a `GOOGLE_VISION_KEY_PATH` variable and there is a `google-vision-key.json` placeholder, but **no code in `tickets.js` ever calls the Vision API**. The billing account issue means: even if the code were wired up, every Vision API call would return a 403 "billing not enabled" error. This is a **known unresolved issue** — see Section 12 Known Issues.

The Tesseract fallback pipeline (`backend/routes/tickets.js` · Lines 401–453): When OCR.Space fails or is rate-limited, the backend:

1. Upscales the image to at least 2400px using `sharp`
2. Runs an **HSV watermark removal** pass — identifies red/pink pixels (hue < 30° or > 330°, saturation > 0.18) and replaces them with white
3. Converts to greyscale, normalises contrast
4. Feeds the cleaned image to Tesseract with `tessedit_pageseg_mode: 3` (auto page segmentation)

The HSV removal works for most tickets but is imperfect — when the watermark overlaps a number printed in a similar hue range, that number's pixels can also be whitened out, making it invisible to Tesseract. This is what caused C. 5089 to be lost in the example above.

🔑 **Heart of the Logic** — ``backend/routes/tickets.js`` · Lines 17 – 300 ·
``parseTicketText()``

What it does: Takes raw OCR text from the ticket image and extracts all structured data: game type, draw date, numbers, bet type, system size. Every field that gets saved to Firestore originates here.

How to navigate to this code:

1. Open `backend/routes/tickets.js`
 2. Press `Ctrl+G` → type `17` → Enter — OR — search for `function parseTicketText`
-

🔑 **Key Lines** — ``backend/routes/tickets.js`` · Lines 23 – 24 · **Game type detection**

What it does: Decides whether the ticket is 4D or TOTO. This single check controls the entire parsing path downstream. TOTO tickets always contain the word "TOTO" printed on them; 4D tickets do not.

How to navigate:

1. Open `backend/routes/tickets.js`
 2. Press `Ctrl+G` → type `23` → Enter — OR — search for `const isTOTO`
-

```
// — Lines 23-24 —  
const isTOTO = upper.includes('TOTO'); // TOTO tickets always print "TOTO"  
const gameType = isTOTO ? 'TOTO' : '4D'; // everything else is 4D
```

🔑 **Key Lines — `backend/routes/tickets.js` · Lines 100 – 107 · 4D number extraction (alphabet loop)**

What it does: Extracts 4-digit numbers from the OCR text by iterating A→Z. Singapore Pools tickets always label numbers as "A. 5888", "B. 4392", etc. This loop finds each letter-labelled number in the correct order, even if the OCR output has extra spaces, dots, or noise between the letter and the number.

How to navigate:

1. Open `backend/routes/tickets.js`
2. Press `Ctrl+G` → type `100` → Enter — OR — search for `for (let code = 65`

4D number extraction — primary method (lines 89–96):

```
// Singapore Pools tickets label numbers alphabetically: A. 5888 B. 4392 C. 1234
// We iterate A-Z and look for letter + optional separators + 4 digits
for (let code = 65; code <= 90; code++) { // A = 65, Z = 90 in ASCII
  const letter = String.fromCharCode(code);
  const re = new RegExp(`(?:^|[\s,;])${letter}[\s.:]{0,5}(\d{4})\b`, 'gm');
  for (const m of searchText.matchAll(re)) {
    if (!seen.has(m[1])) { seen.add(m[1]); combinations.push(m[1]); }
  }
}
```

Why this works: On a real Singapore Pools ticket, numbers are always labelled "A.", "B.", "C." in order. By iterating the alphabet we capture them in the correct order, even if the OCR output has extra whitespace or noise.

🔑 Key Lines — `backend/routes/tickets.js` · Lines 207 – 212 · TOTO System Bet detection

What it does: Detects whether a TOTO ticket is a System Bet (e.g. "SYSTEM 7"). After detection, `expandSystemBet()` generates all C(n,6) combinations automatically: System 7 = 7 combinations, System 12 = 924 combinations. The user only buys one ticket but every combination is checked against the draw result.

How to navigate:

1. Open `backend/routes/tickets.js`
 2. Press `Ctrl+G` → type `207` → Enter — OR — search for `SYSTEM\s+`
-

```
// — Lines 207-212 —  
const sysMatch = rawText.match(/System\s*(7|8|9|10|11|12)\b/i);  
if (sysMatch) {  
  systemSize = parseInt(sysMatch[1]); // e.g. 7, 8, 9, 10, 11, or 12  
  betType    = `System ${systemSize}`;  
}
```

🔑 Key Lines — `backend/routes/tickets.js` · Lines 346 – 365 · OCR.Space API call (primary OCR)

What it does: Sends the ticket image to the OCR.Space cloud API and receives raw text. This is the primary OCR engine — faster and more accurate than local Tesseract. The `'helloworld'` key is the free public demo key. For production, register at ocr.space.

How to navigate:

1. Open `backend/routes/tickets.js`
 2. Press `Ctrl+G` → type `346` → Enter — OR — search for `api.ocr.space`
-

OCR.Space API call — primary OCR engine (actual code in `backend/routes/tickets.js`):

```
// Image is sharpened + normalised first to improve watermark handling
const ocrImageBuffer = await sharp(req.file.buffer)
  .resize({ width: 1600, height: 1600, fit: 'inside', withoutEnlargement: true })
  .sharpen({ sigma: 1.5 })
  .normalise()
  .jpeg({ quality: 92 })
  .toBuffer();

const base64Image = `data:image/jpeg;base64,${ocrImageBuffer.toString('base64')}`;

// Sent as base64 via URLSearchParams (no multipart, avoids form-data size limits)
const body = new URLSearchParams({
  base64Image,
  language:      'eng',
  OCREngine:     '2',          // Engine 2 = OCR.Space's own model (more accurate)
  scale:         'true',
  isTable:       'false',
  detectOrientation: 'true',
});

const ocrRes = await fetch('https://api.ocr.space/parse/image', {
  method: 'POST',
  headers: {
    apikey:      process.env.OCRSPACE_API_KEY || 'helloworld',
    'Content-Type': 'application/x-www-form-urlencoded',
  },
  body: body.toString(),
});

const ocrData = await ocrRes.json();
const rawText = ocrData.ParsedResults?.[0]?.ParsedText?.trim() || '';
```

'helloworld' is OCR.Space's public demo key — 500 requests/month free. Set `OCRSPACE_API_KEY` in `backend/.env` with a registered key for higher limits. The engine tries Engine 2 first, falls back to Engine 1, then Tesseract.

🔑 **Key Lines** — ``backend/routes/tickets.js`` · Lines 449 – 453 · Tesseract fallback (local OCR)

What it does: If OCR.Space fails or returns empty text, the backend falls back to Tesseract.js running locally on the server. No internet needed for this fallback. Accuracy is lower than OCR.Space, but it keeps the upload working offline.

How to navigate:

1. Open `backend/routes/tickets.js`
 2. Press `Ctrl+G` → type `449` → Enter — OR — search for `Tesseract.createWorker`
-

```
// Actual Tesseract call (backend/routes/tickets.js)
const worker = await Tesseract.createWorker('eng');
await worker.setParameters({ tessedit_pageseg_mode: '3' }); // auto page segmentation
const { data: { text: tesseractText } } = await worker.recognize(processedBuffer);
await worker.terminate();
text = tesseractText;
```

🔑 **Key Lines** — ``backend/routes/tickets.js`` · Lines 514 – 530 · Firestore save

What it does: Saves the fully parsed ticket as a Firestore document. `drawType` and `resultStatus` are the two fields the cron job queries later. Every pending future-draw ticket has `resultStatus: 'pending'` until the cron job updates it.

How to navigate:

1. Open `backend/routes/tickets.js`
 2. Press `Ctrl+G` → type `514` → Enter — OR — search for `db.collection('tickets').add`
-

Firestore save (lines 460–475):

```
const ticketData = {
  gameType, drawDate, numbers: combinations,
  betType, systemSize, combinationCount,
  expandedCombinations, // null for ordinary bets
  amount, serialNumber,
  drawType: classifyDrawDate(drawDate), // 'past' or 'future'
  resultStatus: 'pending', // updated later by cron job
  uploadedAt: admin.firestore.FieldValue.serverTimestamp(),
  userId: req.user?.uid || null,
};
const docRef = await db.collection('tickets').add(ticketData);
```

Key Lines — ``backend/routes/tickets.js`` · Lines 534 – 555 · Immediate past-draw check

What it does: If the ticket's draw date has already passed, the backend skips the cron job entirely and checks the result right now, during the upload request. This means the user sees a win/loss result instantly after scanning an old ticket.

How to navigate:

1. Open `backend/routes/tickets.js`
2. Press `Ctrl+G` → type `534` → Enter — OR — search for `if (drawType === 'past')`

Immediate past-draw result check (lines 480–503): If the draw date is already in the past, the backend doesn't wait for the cron job — it checks immediately during the upload request:

```

if (drawType === 'past') {
  const official = gameType === '4D'
    ? await scrape4DResults(drawDate)
    : await scrapeTOTOTResults(drawDate);

  const comparison = gameType === '4D'
    ? compare4DTicket(ticketData, official)
    : compareTOTOTicket(ticketData, official);

  // Update Firestore with result
  await docRef.update({
    resultStatus: comparison.won ? 'won' : 'not_won',
    prizeTier: comparison.prizeTier || null,
    winMatches: comparison.matches || [],
  });
  // Return result immediately in the API response
  responseData.comparison = comparison;
}

```

7.4 Backend: Result Checking + Notifications

File: `backend/server.js`

This file is the entry point for the entire backend. It does four things:

1. **Sets up Express routes** — maps URL paths to their handler files
2. **Starts cron jobs** — scheduled tasks that run automatically
3. **Seeds mock data on startup** — so the app has results to show immediately
4. **Scrapes live results on startup** — in the background, fetches real Singapore Pools data

Route registration (lines 22–27):

```

app.use('/api/auth', require('./routes/auth'));
app.use('/api/tickets', require('./routes/tickets'));
app.use('/api/results', require('./routes/results'));
app.use('/api/predict', require('./routes/predict'));
app.use('/api/devices', require('./routes/devices'));

```

```
[4D parsed date] 24/07/19 → Draw No. 3482
[scraper] No 4D result found for 24/07/19

C:\Users\Valentine Virgo\Documents\4d-toto-app\backend>node server.js
[dotenv@17.3.1] injecting env (5) from .env -- tip: enable debug logging with { debug: true }
[SG] Lottery Backend running on port 3001
Cron jobs: hourly result polling + daily result refresh
[scraper] Regenerating mock history with correct draw numbers...
[scraper] 4D & TOTO mock history regenerated.
[startup] Scraping latest results from Singapore Pools...
[4D page title] Singaporepools - Latest 4D Results
[4D nums found] [ '941', '9874', '1489', '0412', '1147' ]
[4D parsed date] 15/03/2026 → Draw No. 4522
[TOTO page title] Singaporepools - TOTO Results
[TOTO nums found] [
  12, 25, 33, 40,
  43, 46, 21, 1
]

[TOTO parsed date] 12/03/2026 → Draw No. 3928
[scraper] TOTO live scrape success: 12/03/2026
[startup] TOTO scraped: 12/03/2026 Draw 3928 (live)
[scraper] 4D live scrape success: 15/03/2026
[startup] 4D scraped: 15/03/2026 Draw 4522 (live)
[upload] Request received
[upload] OCR via OCR.Space, length: 306
[upload] OCR engine: ocr.space, text length: 306
[upload] OCR complete, text length: 306
[upload] OCR raw text:
---
APORE
4D-
o L S
o L S
SINGAPORE
ORDINARY
SINGAPORE
SINGAPORE
GAPORE
S
A 5888
BIG $20
SWL $30
S
4392
```

Backend terminal output

🔑 Heart of the Logic — `backend/server.js` · Lines 50 – 124 · Hourly cron job

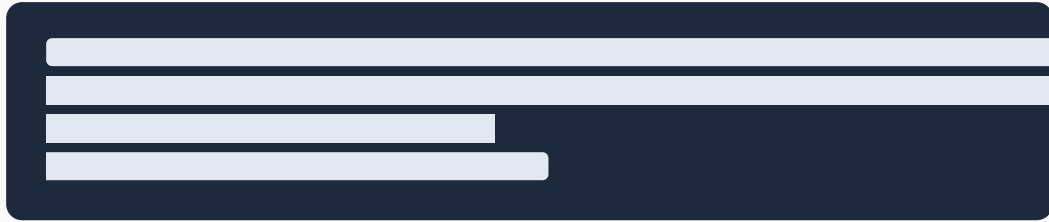
What it does: This is what makes the app "smart" — it runs automatically every hour. It finds all tickets marked `pending` whose draw date has now passed, scrapes the official Singapore Pools result, compares the numbers, updates Firestore with won/not_won, and sends a push notification to all devices. Without this cron job, users would never find out if they won.

How to navigate:

1. Open `backend/server.js`
2. Press `Ctrl+G` → type `50` → Enter — OR — search for `cron.schedule('0 * * * *')`

Cron syntax quick reference — for interns who haven't seen cron before:

A cron expression has 5 fields: `minute hour day-of-month month day-of-week` An asterisk `*` means "every" for that field.



A useful free tool to decode any cron string: crontab.guru (<https://crontab.guru>).

Heart of the logic — hourly cron job (lines 50–124):

This is what makes the app "smart". Every hour, it checks all tickets whose draw date has now passed:

```

cron.schedule('0 * * * *', async () => {
  // Find all tickets that are still 'pending' and were future-draw tickets
  const snap = await db.collection('tickets')
    .where('resultStatus', '==', 'pending')
    .where('drawType', '==', 'future')
    .get();

  for (const doc of snap.docs) {
    const ticket = doc.data();

    // Skip if the draw is still in the future
    if (classifyDrawDate(ticket.drawDate) === 'future') continue;

    // Scrape official result from Singapore Pools
    const official = ticket.gameType === '4D'
      ? await scrape4DResults(ticket.drawDate)
      : await scrapeTOTOResults(ticket.drawDate);

    // Compare ticket numbers against official result
    const comparison = ticket.gameType === '4D'
      ? compare4DTicket(ticket, official)
      : compareTOTOTicket(ticket, official);

    // Update Firestore with win/loss result
    await doc.ref.update({
      resultStatus: comparison.won ? 'won' : 'not_won',
      prizeTier: comparison.prizeTier || null,
      // Also save notification data for client polling (web fallback)
      notification: {
        title: comparison.won ? '🎉 You won!' : 'Draw result available',
        body: comparison.won ? `Won ${comparison.prizeTier}!` : 'Did not win.',
        won: comparison.won,
        read: false,
      },
    });

    // Send real push notification to all registered mobile devices
    await sendPushToAll(pushTitle, pushBody, { ticketId: doc.id, won: comparison.won });
  }
});

```

🔑 Secondary Logic — `backend/server.js` · Lines 128 – 136 · Daily results refresh

What it does: Every night at 11 PM, scrapes the latest 4D and TOTO results to keep the Results page up to date even if no tickets were uploaded that day.

How to navigate:

1. Open `backend/server.js`
2. Press `Ctrl+G` → type `128` → Enter — OR — search for `cron.schedule('0 23`

```
// — Lines 128-136 —  
cron.schedule('0 23 * * *', async () => {  
  await Promise.all([scrape4DResults(null), scrapeTOTOResults(null)]);  
});
```

7.5 Results Scraper

File: `backend/services/scrapper.js`

Responsible for getting official draw results from Singapore Pools.

What is Puppeteer / web scraping? — for interns new to this concept:

Puppeteer launches a **real Chrome browser invisibly** (no window, no screen — called "headless" mode). It is not an API call. It literally opens the Singapore Pools website the same way you would in a browser, waits for the page to fully load, then reads the HTML to extract the numbers.

Why not use an API? Singapore Pools does not provide a public results API. Scraping is the only option.

Why it can break: If Singapore Pools changes their website layout — even just renaming a CSS class or restructuring a `<div>` — the selectors we use to find the numbers (`td` , `span` , etc.) may stop matching. When this happens, the scraper returns empty results and the app falls back to mock data. The fix is to inspect the new Singapore Pools page in Chrome DevTools and update the selectors in `scraper.js` .

Three-layer fallback system:

1. Check Firestore cache (fastest, avoids repeated scraping)
 - found? return cached result
 - ↓ not found
2. Puppeteer: launch headless Chrome, navigate to SP website, extract numbers
 - success? save to Firestore cache, return result
 - ↓ failed (SP blocked, timeout, etc.)
3. Mock data: generate realistic-looking random numbers for demo/dev
 - return mock result (marked source: 'mock')

Past Results

Official Singapore Pools winning numbers

4DTOTO

SEARCH BY DRAW DATE

DD/MM/YY e.g. 15/03/26

Q Search

Showing last 7 days by default - Search any date to look up a specific draw

LAST 7 DAYS

Refresh

DRAW DATE	DRAW NO.	1ST PRIZE	2ND PRIZE	3RD PRIZE
LATEST Sun, 15 Mar 2026	#4522	5747	9874	1497
STARTER PRIZES (10)				
0412 1147 1243 1985 2179 3468 6074 7024 8159 9891				
CONSOLATION PRIZES (10)				
0691 2049 3591 3811 6162 6311 6915 8456 8658 9061				
Sat, 14 Mar 2026	#4521	6483	0288	8484
Wed, 11 Mar 2026	#4520	4400	7690	4742

4D Results Page

🔑 Heart of the Logic — `backend/services/scrapper.js` · Lines 84 – 101 · Puppeteer DOM extraction

What it does: Launches a headless Chrome browser, opens the Singapore Pools website, then scans every DOM element for text that is exactly a 4-digit number. The first 3 unique results = 1st/2nd/3rd prize. Next 10 = Starters. Next 10 = Consolation. This relies on the order Singapore Pools renders numbers on their page.

How to navigate:

1. Open `backend/services/scrapper.js`
2. Press `Ctrl+G` → type `84` → Enter — OR — search for `page.evaluate`

Heart of the logic — Puppeteer 4D scrape (lines 84–101):

```
const data = await page.evaluate(() => {
  const seen = new Set();
  const arr = [];
  // Scan every DOM element for text that looks exactly like a 4-digit number
  document.querySelectorAll('td, span, div, li, p, strong, b').forEach(el => {
    const t = el.textContent.trim();
    if (/^\d{4}$/.test(t) && !seen.has(t)) { seen.add(t); arr.push(t); }
  });
  return arr;
});
// First 3 unique 4-digit numbers = 1st, 2nd, 3rd prize
// Next 10 = Starter prizes, next 10 = Consolation prizes
return {
  first:      data.nums[0],
  second:     data.nums[1],
  third:      data.nums[2],
  starters:   data.nums.slice(3, 13),
  consolation: data.nums.slice(13, 23),
};
```

Important: "Next Draw Date" problem (lines 50–62): Singapore Pools always shows "Next Draw Date" at the top of the page before the actual "Draw Date". The `extractLatestDrawDate()` function skips any date that is in the future, so it always returns the most recent actual draw date.

🔑 **Key Lines** — ``backend/services/scrapper.js`` · **Lines 482 – 495** · **Draw number estimation**

What it does: Singapore Pools doesn't expose draw numbers cleanly via scraping, so we calculate them from a verified anchor point (Draw 4522 = 15/03/2026). We count how many actual draw days (Wed/Sat/Sun for 4D, Mon/Thu for TOTO) fall between the anchor date and the target date. Simple but precise.

How to navigate:

1. Open `backend/services/scrapper.js`
 2. Press `Ctrl+G` → type `482` → Enter — OR — search for `function estimate4DDrawNumber`
-

Draw number estimation (lines 482–495): Singapore Pools doesn't easily expose draw numbers via scraping, so we calculate them mathematically. We use an anchor point (a known draw number on a known date) and count forward/backward through actual draw days:

```
// Anchor: Draw 4522 = Saturday 15/03/2026 (verified from live scrape)
function estimate4DDrawNumber(drawDateStr, baseDrawNo = 4522, baseDateStr = '15/03',
    // Count how many Wed/Sat/Sun days are between baseDateStr and drawDateStr
    return String(baseDrawNo + count4DDrawDays(base, t));
}
```

7.6 Prize Matching Logic

File: `backend/services/matcher.js`

Compares a ticket's numbers against an official draw result and returns the prize tier.

4D Upload Ticket

Drag & drop, browse, or take a photo



4D **Serial: 846367-8-9785369-0045446**

Already saved

Scan Another

Review your scanned ticket details below.

GAME TYPE	DRAW DATE	BET TYPE
4D	21/02/26	Ordinary
YOUR NUMBERS		
5888 4392		
Ticket ID: 31VklQwc37K135kdJchp		

Upload result card showing correctly parsed 4D ticket — 5888 and 4392, draw date 21/02/26

Above: the result card shown to the user after a successful scan. The serial number, game type, draw date, bet type, and all numbers are extracted correctly. After this point, the backend calls `compare4DTicket()` against the official draw result.

🔑 **Heart of the Logic** — ``backend/services/matcher.js`` · **Lines 49 – 82** · ``compare4DTicket()``

What it does: The core win/loss decision for 4D tickets. For each number on the ticket, it generates the full set of candidates to check: Ordinary = exact match only · iBet = all permutations (up to 24) · System Roll = 10 candidates (digit 0–9). Then checks each candidate against 1st / 2nd / 3rd / Starter / Consolation prize numbers.

How to navigate:

1. Open `backend/services/matcher.js`
 2. Press `Ctrl+G` → type `49` → Enter — OR — search for `function compare4DTicket`
-

What is iBet? — Singapore Pools domain knowledge for interns:

iBet is a Singapore Pools bet type where **any arrangement of your 4 digits wins**, not just an exact match. For example, if you bet on **1234 with iBet**, you also win if the official result is **4321, 2143, 3412**, or any other permutation.

Mathematically, 4 unique digits have **4! = 24 permutations**. But if digits repeat, there are fewer unique arrangements:

- `1234` → 24 permutations (all digits different)
- `1123` → 12 permutations (one pair repeated)
- `1122` → 6 permutations (two pairs repeated)
- `1111` → 1 permutation (all same — effectively a normal bet)

This is exactly why `permutations4D(num)` generates a variable number of candidates and uses a `Set` to deduplicate them before checking.

System Roll is different: one digit on your ticket is printed as "R" (Roll), meaning Singapore Pools checks all 10 possibilities for that position (0–9). So "123R" becomes "1230", "1231", ... "1239" — always exactly 10 candidates.

Heart of the logic — 4D matching (lines 49–82):

```
function compare4DTicket(ticket, result) {
  const numbers = ticket.numbers || [];
  const betType = ticket.betType || 'Ordinary';

  for (const num of numbers) {
    let candidates;

    if (betType === 'iBet') {
      // iBet: any permutation of the 4 digits wins
      // e.g. "1234" also wins if the result is "4321", "2143", etc.
      candidates = permutations4D(num); // generates up to 24 permutations
    } else if (betType === 'System Roll') {
      // System Roll: one digit is replaced by 0-9 ("123R" → "1230","1231"... "1239")
      candidates = expandSystemRoll(num); // generates 10 candidates
    } else {
      // Ordinary bet: must match exactly
      candidates = [String(num).padStart(4, '0')];
    }

    for (const c of candidates) {
      const tier = check4DNumber(c, result); // is c == 1st/2nd/3rd/Starter/Console
      if (tier) {
        matches.push({ number: num, matched: c, prize: tier });
        break; // only one prize per number entry
      }
    }
  }

  return { won: matches.length > 0, matches, prizeTier: matches[0]?.prize };
}
```

Official Singapore Pools winning numbers

4D TOTO

SEARCH BY DRAW DATE

DD/MM/YY e.g. 15/03/26 Q Search

Showing last 7 days by default. Search any date to look up a specific draw

LAST 7 DAYS Refresh

	DRAW DATE	DRAW NO.	WINNING NUMBERS	ADD'L
LATEST	Mon, 16 Mar 2026	#3929	18 19 22 26 38 46	32
	Thu, 12 Mar 2026	#3928	12 25 33 40 43 46	21

TOTO Results Page

🔑 **Key Lines** — ``backend/services/matcher.js`` · **Lines 111 – 128** · **TOTO prize groups**

What it does: Determines the TOTO prize tier for a single 6-number combination. Counts how many of the 6 numbers match the winning draw, plus whether the additional number matches. The 7 prize groups map directly to Singapore Pools official rules.

How to navigate:

1. Open `backend/services/matcher.js`
 2. Press `Ctrl+G` → type `111` → Enter — OR — search for `function checkTOTOCombination`
-

TOTO prize groups (lines 111–128):

```
function checkTOTOCombination(combo, result) {
  const matchWin = combo.filter(n => result.winningNums.includes(n)).length;
  const matchAdd1 = combo.includes(result.add1Num);

  if (matchWin === 6)          return 'Group 1 (Jackpot)';
  if (matchWin === 5 && matchAdd1) return 'Group 2';
  if (matchWin === 5)          return 'Group 3';
  if (matchWin === 4 && matchAdd1) return 'Group 4';
  if (matchWin === 4)          return 'Group 5';
  if (matchWin === 3 && matchAdd1) return 'Group 6';
  if (matchWin === 3)          return 'Group 7';
  return null; // no win
}
```

🔑 **Key Lines** — ``backend/services/matcher.js` · Lines 181 – 198 · classifyDrawDate()`
— **26-hour buffer**

What it does: Decides whether a draw date is 'past' or 'future'. **Critical detail:** It adds a 26-hour buffer — results are not published at midnight but at ~6:30 PM (4D) and ~9:30 PM (TOTO) SGT. Without this buffer, a ticket uploaded on the morning of draw day would be wrongly checked before results exist, always returning "not found".

How to navigate:

1. Open `backend/services/matcher.js`
 2. Press `Ctrl+G` → type `181` → Enter — OR — search for `function classifyDrawDate`
-

Draw date classification (lines 181–198): The `classifyDrawDate()` function adds a 26-hour buffer to the draw date before deciding it's "past". This is because Singapore Pools publishes results at ~6:30 PM (4D) and ~9:30 PM (TOTO), not at midnight. Without this buffer, a morning check on draw day would incorrectly classify the ticket as "past" when results aren't out yet.

7.7 History Page

Web: `web-app/src/views/history/HistoryView.jsx` **Mobile:** `mobile-app/src/views/history/HistoryView.tsx`

Both history pages follow the same pattern:

1. Load all tickets from Firestore via the TicketPresenter
2. Show filter tabs (All / 4D / TOTO / Won)
3. Show each ticket as an expandable card
4. Tapping a ticket reveals full details (numbers, draw date, result)

The page does not fetch data itself — data comes from `TicketPresenter` via React Context. This is the VIPER pattern in action.

7.8 Mobile App

File: `mobile-app/app/_layout.tsx` — Root layout



Unmatched Route

Page could not be found.

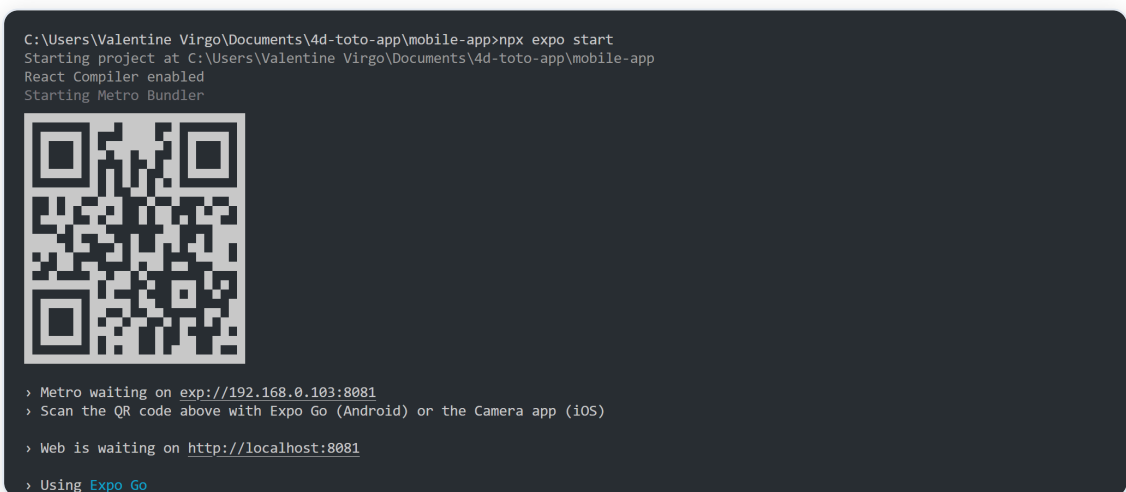
exp://192.168.0.103:8081/--/

[Go back](#) • [Sitemap](#)



Mobile Upload Screen

The mobile Upload screen (shown above) presents two options: **Take Photo** (launches the device camera) and **Choose from Gallery** (opens the photo library). The bottom tab bar shows Home, Upload (active, highlighted in cyan), and Explore. Notice the Android gesture navigation bar at the very bottom — this is exactly why `useSafeAreaInsets()` is needed to prevent the tab bar from overlapping it.



Expo Dev Build

🔑 Heart of the Logic — `mobile-app/app/\_layout.tsx` · Lines 18 – 40 · Root wrapper order

What it does: Defines the exact nesting order of required providers. The order matters — each layer depends on the one outside it. `GestureHandlerRootView` must be outermost or swipe gestures crash. `SafeAreaProvider` must wrap everything that calls `useSafeAreaInsets()`. Presenters load all data once here so every tab shares it without re-fetching.

How to navigate:

1. Open `mobile-app/app/_layout.tsx`
2. Press `Ctrl+G` → type `18` → Enter — OR — search for `GestureHandlerRootView`

This is the outermost wrapper for the entire mobile app. It must include:

```
<GestureHandlerRootView style={{ flex: 1 }}> ← Required for swipe/gesture features
  <SafeAreaProvider>                             ← Required for edge-to-edge display
    <ThemeProvider value={DarkTheme}>
      <TicketPresenter>                          ← Loads ticket data
        <ResultsPresenter>                      ← Loads results data
          <Stack>
            <Stack.Screen name="index" />        ← Redirect screen (→ /tabs)
            <Stack.Screen name="tabs" />        ← Main tab navigator
          </Stack>
        </ResultsPresenter>
      </TicketPresenter>
    </ThemeProvider>
  </SafeAreaProvider>
</GestureHandlerRootView>
```

Why `GestureHandlerRootView`? The History page uses `Swipeable` (from `react-native-gesture-handler`) to allow swipe-to-delete on tickets. This component requires `GestureHandlerRootView` to be present at the root of the app. Without it, the app crashes with a `PanGestureHandler` error.

Why SafeAreaProvider? Android 10+ uses edge-to-edge display (content goes behind the navigation buttons at the bottom). `SafeAreaProvider` + `useSafeAreaInsets()` lets us add the correct amount of padding so buttons aren't hidden behind the phone's navigation bar.

File: `mobile-app/app/tabs/_layout.tsx` — Tab bar

Defines the 3 main tabs: Home, Scan (Upload), History. The tab bar height is dynamic:

```
const insets = useSafeAreaInsets();
tabBarStyle: {
  height: 60 + insets.bottom, // Add system navigation bar height
  paddingBottom: insets.bottom + 6, // Prevent icons being hidden
}
```

File: `mobile-app/hooks/useNotifications.ts` — Push notifications

🔑 **Heart of the Logic** — `mobile-app/hooks/useNotifications.ts` · Lines 1 – 60 ·
Device registration + polling

What it does: Two jobs in one hook. (1) On first load, gets the device's Expo push token and saves it to the backend so the cron job knows where to send notifications when a ticket wins. (2) Every 60 seconds, polls Firestore for any unread win/loss notifications and fires a local notification — this is the fallback for Expo Go where real push doesn't work.

How to navigate:

1. Open `mobile-app/hooks/useNotifications.ts`
 2. Press `Ctrl+G` → type `1` → Enter — the whole file is this hook — OR — search for `registerAndSaveToken` (device registration) — OR — search for `checkForNotifs` (polling fallback)
-

This hook runs once when the app loads. It does two things:

1. Registers the device for push notifications and saves the Expo push token to the backend
2. Starts a polling loop (every 60 seconds) that checks Firestore for new win/loss notifications

```
// Polling fallback (works in Expo Go)
const checkForNotifs = async () => {
  const snap = await db.collection('tickets')
    .where('notification.read', '==', false)
    .get();
  for (const doc of snap.docs) {
    Notifications.scheduleNotificationAsync({
      content: { title: doc.data().notification.title, body: doc.data().notification.body },
      trigger: null, // show immediately
    });
    await doc.ref.update({ 'notification.read': true });
  }
};
```

8. Feature Guide

Uploading a Ticket

Web:

1. Click "Upload Ticket" in the sidebar
2. Drag & drop your ticket photo, click "Browse Files", or click "Use Camera"
3. Wait for OCR to process (usually 3–8 seconds)
4. Review extracted numbers, draw date, and bet type
5. The result (won / not won / pending) is shown automatically

Mobile:

1. Tap the camera icon in the bottom tab bar (Scan tab)
2. Tap "Camera" to take a photo or "Gallery" to choose existing
3. The same OCR processing happens and result is displayed

Supported ticket types:

- 4D Ordinary (Big/Small)
- 4D iBet
- 4D System Bet (System 5, 6, 7)
- 4D System Roll
- TOTO Ordinary
- TOTO System Bet (System 7–12)
- TOTO Quick Pick

- TOTO iTOTO

Viewing Draw Results

1. Click/tap "Results" in the navigation
2. Toggle between 4D and TOTO tabs
3. Latest draw is shown at the top
4. Scroll down for previous draws

Results are sourced from Singapore Pools (live scrape). If the live scrape fails (e.g., the SP website is down), the app shows mock results clearly labelled "Mock Data".

Ticket History

1. Click/tap "History" in the navigation
2. Use the filter tabs to show All / 4D / TOTO / Won tickets
3. Click/tap a ticket to expand its full details
4. Swipe left on a ticket (mobile) or click the delete icon to remove it

Push Notifications (Web)

When you upload a ticket for a future draw:

1. A prompt appears: "Enable Notifications"
2. Click it and allow notifications in your browser
3. The app will check for results every 60 seconds while the browser tab is open
4. When results are available, a browser notification is fired

Push Notifications (Mobile)

On the development build (APK), real push notifications are supported:

1. The app registers your device's Expo push token when it first opens
2. This token is saved to Firestore `devices` collection
3. When the backend cron job finds a result for a pending ticket, it sends a push notification to all registered devices via Expo's push service

9. Predictive Analysis Guide

File: `backend/routes/predict.js` **Page:** `web-app/src/pages/PredictPage.jsx`

The Predict page analyses past draw results and suggests numbers using three statistical models. These are mathematical observations, not guarantees — lottery draws are random.

Where does the historical data come from? The `/api/predict` endpoint calls `getPast4DResults(50)` and `getPastTOTOResults(100)` from `scraper.js` — these read from the `results_4d` and `results_toto` Firestore collections (populated nightly by the scraper). The data is passed to `generatePredictions()` in `backend/services/predictor.js` which runs all three models.

What the endpoint actually returns (`GET /api/predict`):



The frontend iterates over `predictions` and renders one card per model.

Model 1: Digit Frequency Analysis (`modelId: "frequency"`)

File: `backend/services/predictor.js` • `runDigitFrequencyModel()`

For **4D**: analyses each of the four digit positions (1st, 2nd, 3rd, 4th digit) independently. It counts how often each digit 0–9 has appeared at each position across past draws, weighted by recency using exponential decay ($e^{-\lambda \cdot i}$ — more recent draws count more). The most frequent digit per position is combined to produce one 4-digit prediction.

For **TOTO**: counts how often each number 1–49 has appeared across all past draws (also recency-weighted) and selects the top 12 most frequent.

Model 2: Hot & Cold Number Analysis (`modelId: "hot_cold"`)

File: `backend/services/predictor.js` • `runHotColdModel()`

Defines **Hot** numbers as those drawn in the last 10 TOTO draws, and **Cold** numbers as those drawn 10–30 draws ago but absent from the last 10. The System 12 prediction uses the top-6 Hot numbers as the primary selection and top-6 Cold numbers as the supplementary selection — covering both extremes.

For **4D**: the most frequently repeated 4-digit combination across the last 20 draws is used. If no number repeats, the most recent 1st prize is used.

Important statistical note for interns — read before explaining this feature to users: Each lottery draw is **statistically independent**. The Singapore Pools machine has no memory of previous draws. A number that hasn't appeared in 50 draws is not "due" — its probability for the next draw is identical to every other number. The "Cold" concept in Model 2 is explicitly the **Gambler's Fallacy**. **These models are provided for interest and engagement, not as predictive tools.** Always be honest with users: no algorithm can predict a fair random draw.

Model 3: Odd & Even / Jackpot Pattern Analysis (`modelId: "odd\_even"`)

File: `backend/services/predictor.js` • `runOddEvenModel()`

For **4D**: finds the most common odd/even pattern across all digit positions (e.g., OEOE = odd-even-odd-even). Then for each position picks the most frequent digit of the required parity.

For **TOTO**: analyses the mathematical profile of historical jackpot draws — the odd/even balance, the low (1–24) vs high (25–49) split, and the average sum. Generates 12 numbers that match the most common historical profile.

How to Use the Predict Page

1. Navigate to **Predict** in the sidebar (web) or tab bar (mobile)
2. The page automatically runs all three models on both 4D and TOTO at once — no input needed
3. Each model shows a predicted 4D number and a predicted TOTO set of 12 numbers
4. A `confidenceScore` is shown (e.g. `0.12`) — this reflects model certainty, not win probability
5. Use these as a starting point — combine with your own judgement

Note for interns: There is no "number of draws" selector in the UI. The backend always analyses the last 50 past 4D draws and last 100 past TOTO draws automatically. If you want to change this, edit the constants at the top of `backend/services/predictor.js`.

10. Testing Each Feature

Manual Testing Checklist

Backend (test with curl or Postman):

```
# Health check
curl http://localhost:3001/

# Upload a ticket (replace path with an actual ticket image)
curl -X POST http://localhost:3001/api/tickets/upload \
  -F "ticket=@/path/to/ticket.jpg"

# List all tickets
curl http://localhost:3001/api/tickets

# Get latest 4D results
curl http://localhost:3001/api/results/4d

# Get latest TOTO results
curl http://localhost:3001/api/results/toto

# Run prediction analysis (runs all 3 models on both 4D and TOTO at once – no query)
curl http://localhost:3001/api/predict
```

Web App:

- ☐ Home page loads without errors
- ☐ Drag and drop a ticket image → OCR result appears
- ☐ File browse button opens file picker
- ☐ Camera button opens camera (on mobile browser)
- ☐ Past-draw ticket shows win/loss immediately
- ☐ Future-draw ticket shows "Draw Result Pending" with timeline
- ☐ History page loads and lists tickets
- ☐ Filter tabs work (All / 4D / TOTO / Won)
- ☐ Results page shows 4D and TOTO draws
- ☐ Predict page generates suggestions
- ☐ Delete a ticket from history

Mobile App:

- ☐ App opens without crash

- ☐ All 3 tabs accessible (Home, Scan, History)
- ☐ Tab bar not hidden behind phone navigation buttons
- ☐ Camera/gallery upload works
- ☐ OCR result displays correctly
- ☐ Settings tab accessible without crash
- ☐ Swipe to delete ticket in history

11. Troubleshooting

Backend will not start

Symptom: `Error: Cannot find module './firebase'` **Fix:** Make sure `serviceAccountKey.json` exists in the `backend/` folder. See [Step 2 of Installation](#).

Symptom: `FirebaseError: Could not load the default credentials` **Fix:** Your `.env` file is missing or the `FIREBASE_PRIVATE_KEY` value is incorrect. The private key must include the full `-----BEGIN PRIVATE KEY-----` header and footer.

Symptom: `Port 3001 is already in use` **Fix:** Kill the existing process:

```
# On Mac/Linux:
lsof -ti:3001 | xargs kill
# On Windows:
netstat -ano | findstr :3001
taskkill /PID <PID> /F
```

OCR returns no numbers

Possible causes:

1. Image is blurry or too dark — try a clearer photo with better lighting
2. OCR.Space API is rate-limited — the free `helloworld` key allows ~500 requests/month. When exhausted, the app falls back to Tesseract. Check the backend console: `[upload] OCR.Space unavailable, falling back to Tesseract`
3. Ticket format is unusual — if it's a non-standard Singapore Pools format, the parser may not recognise it yet

Debug tip: The backend logs the raw OCR text for every upload. Check the terminal for:

```
[upload] OCR engine: ocr.space-e2, text length: 312
[upload] OCR raw text:
---
4D
ORDINARY
A. 8537 BIG $1 SML $0
B. 9724 BIG $1 SML $0
C. 5089 BIG $1 SML $0
D. 9743 BIG $1 SML $0
PRICE: $4.00
DRAW: WED 24/07/19
---
```

If a number appears in the raw text but not in the extracted result, the bug is in `parseTicketText()`. If a number is missing from the raw text entirely, the bug is in the OCR engine or the image preprocessing.

OCR Known Fault Case — Partial Number Extraction

Symptom: The app successfully uploads a ticket and shows numbers, but fewer numbers appear than are printed on the physical ticket. No error is shown.

4D

Upload Ticket

Drag & drop, browse, or take a photo



4D

Serial: 127-52682448-096708

⚠ Already saved

Scan Another

Review your scanned ticket details below.

GAME TYPE	DRAW DATE	BET TYPE
4D	24/07/19	Ordinary
YOUR NUMBERS		
8537 9724 9743		
Ticket ID: UtvvvqUSN0VsdQ1QXIEL		

OCR Fault Case — C. 5089 dropped

Real example (shown above):

- Physical ticket: A. 8537 · B. 9724 · **C. 5089** · D. 9743 (4 numbers)
- App extracted: 8537, 9724, 9743 (3 numbers — **C. 5089 silently dropped**)

Why this happens: When OCR.Space is unavailable and Tesseract runs instead, the HSV watermark removal step (`backend/routes/tickets.js` lines 401–434) identifies red/pink pixels and whites them out. The Singapore Pools watermark is a red "S" logo printed repeatedly across the ticket. If the "5089" on line C happens to sit on top of a dense patch of this watermark, some of the digits' pixels may also fall within the red hue range and get erased before Tesseract reads the image. The result is that Tesseract sees a blank or partial line C and produces no output for it.

Root cause in the HSV filter (line 434):

```
// Any pixel with hue in 0-30° or 330-360° AND saturation > 0.18 is whitened.  
// Problem: ink printed over red watermark can be partially red-tinted too.  
if ((hue < 30 || hue > 330) && sat > 0.18) {  
  rawPixels[i] = rawPixels[i + 1] = rawPixels[i + 2] = 255; // → white  
}
```

The saturation threshold `0.18` is too generous — it catches not just the watermark but also ticket ink that is slightly warm-toned due to the photo's lighting or the watermark bleeding through.

Current status: Unresolved. OCR.Space handles this correctly (it ignores visual styling and reads text), so the issue only manifests when OCR.Space is unavailable.

Why Google Vision isn't the fix (yet): Google Cloud Vision would solve this permanently — it has the best accuracy on printed text with complex backgrounds. However, the project has a Google Cloud account with a service account key (`GOOGLE_VISION_KEY_PATH` in `.env`) but **billing is not activated** on the account. Google Vision requires billing to be enabled even on the free tier. Until billing is resolved, Google Vision cannot be called — every request returns HTTP 403 with `"BILLING_DISABLED"`. No code currently calls the Vision API for this reason.

Mobile app cannot connect to backend

Symptom: "Network request failed" or "Upload failed. Is the backend running?"

Check:

1. Is the backend running? (`node server.js` should show a success message)
2. Is the backend URL in the mobile app config correct?
3. Are the phone and computer on the same WiFi network?
4. If using ngrok, is the ngrok tunnel still active? (free tunnels expire after 8 hours)
5. When using ngrok, are you sending the `ngrok-skip-browser-warning: true` header?

Settings tab crashes on Android

Symptom: App exits when tapping Settings tab. **Cause:** `SafeAreaView` imported from `'react-native'` instead of `'react-native-safe-area-context'` **Fix:** In `mobile-app/app/tabs/settings.tsx`, change:

```
// WRONG:
import { SafeAreaView } from 'react-native';

// CORRECT:
import { SafeAreaView } from 'react-native-safe-area-context';
```

"Unmatched Route" error on development build

Symptom: Dev build shows "Unmatched Route: /" on startup. **Fix:** Make sure `mobile-app/app/index.tsx` exists with:

```
import { Redirect } from 'expo-router';
export default function Index() {
  return <Redirect href="/tabs" />;
}
```

Tab bar hidden behind phone navigation buttons

Symptom: Bottom tab bar overlaps with Android gesture navigation area. **Fix:** In `mobile-app/app/tabs/_layout.tsx`, use `useSafeAreaInsets()`:

```
const insets = useSafeAreaInsets();
// In tabBarStyle:
height: 60 + insets.bottom,
paddingBottom: insets.bottom + 6,
```

Mobile app fails to connect — "Failed to connect to port 8081"

Symptom: Expo Go on your phone shows `Error Loading App: Failed to connect to /192.168.x.x (port 8081)`.

This is the most common setup issue on Windows. Windows Firewall blocks port 8081 by default, so your phone cannot reach the Expo Metro bundler running on your computer.

Fix A — Add firewall rule (recommended, one-time):

Open **PowerShell as Administrator** (press `Win` → type `PowerShell` → right-click → Run as administrator) and run:

```
netsh advfirewall firewall add rule name="Expo Metro 8081" dir=in action=allow prot
```

Then restart Expo (`npm expo start`) and scan the QR code again.

Fix B — Use tunnel mode (no admin rights needed):

```
cd mobile-app
npx expo start --tunnel
```

This routes through Expo's cloud servers — works across any network, no firewall change needed. Slightly slower initial load.

Fix C — Check same WiFi: Your phone and computer must be on the **same WiFi network**. If your phone is on mobile data, the local IP `192.168.x.x` is unreachable regardless of firewall settings.

Push notifications not arriving (mobile)

Important: Real push notifications require the development build (APK), not Expo Go.

In Expo Go, the polling fallback is used instead:

- The app checks Firestore every 60 seconds while it's open
- If a ticket result has been updated, a local notification is triggered
- This requires the app to be open and in the foreground

For real background push notifications, install the development APK.

Singapore Pools scraping fails

Symptom: Results page shows mock data / `[puppeteer] 4D scrape error` in console.

Possible causes:

1. Singapore Pools changed their website layout — the DOM selectors may need updating
2. Network blocked Puppeteer's headless Chrome — try adding a delay or rotating the user agent
3. Server has no internet access

Temporary fix: Mock data is shown automatically, so the app stays functional. Real results can be entered manually into Firestore if needed.

12. Known Issues

Issue	Severity	Workaround
Tesseract fallback silently drops numbers — when OCR.Space is unavailable, Tesseract's HSV watermark removal can erase digit pixels that overlap the red watermark, causing numbers to be extracted partially (e.g. 3 of 4 numbers). No error is raised.	High	Ensure OCR.Space is reachable and the <code>OCRSPACE_API_KEY</code> env var is set to a registered key (not <code>helloworld</code>) to avoid rate limits hitting the fallback path. See the fault case in Section 11.
Google Vision not wired up — <code>GOOGLE_VISION_KEY_PATH</code> exists in <code>.env</code> but no code calls the Vision API. Billing is not enabled on the Google Cloud account, so it cannot be activated.	High	Intern challenge — see Section 17. Best long-term fix once billing is resolved.
OCR.Space <code>helloworld</code> key rate-limited — free public key allows ~500 requests/month. Once exhausted, every upload falls back to the less accurate Tesseract.	Medium	Register a free OCR.Space account at ocr.space to get a personal key with a higher limit. Set <code>OCRSPACE_API_KEY=your-key</code> in <code>backend/.env</code> .
Firebase Storage not available on Spark plan	Medium	Ticket photos are not stored in cloud storage; only extracted numbers are saved in Firestore
Singapore Pools website may block Puppeteer	Medium	App falls back to mock data automatically
Push notifications don't work in Expo Go	Low	Use development APK build, or rely on polling fallback
TOTO System 12 (924 combos) is slow to process	Low	Processing takes ~1–2 seconds; show a loading indicator
ngrok free plan only allows 1 tunnel	Low	Use same-WiFi connection instead of ngrok for mobile testing

13. Data Privacy & Security

What Data is Stored

Data	Stored Where	Who Can See It
Ticket numbers	Firestore <code>tickets</code>	Authenticated user only
Draw dates	Firestore <code>tickets</code>	Authenticated user only
Ticket photos	NOT stored	Nobody (deleted after OCR)
Push tokens	Firestore <code>devices</code>	Backend only
User email/password	Firebase Auth	Firebase only (hashed)

What is NOT Stored

- Ticket photos are processed in memory and immediately discarded after OCR
- No personal financial information is stored
- No location data

Security Practices

- `serviceAccountKey.json` and `.env` are git-ignored and must never be committed
- The OCR.Space API key is a public demo key — for production, register your own

Firebase Security Rules — restrict read/write to authenticated users only:

Go to console.firebase.google.com (<https://console.firebase.google.com>) → your project → **Firestore Database** → **Rules** tab. Replace the default rules with:

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Tickets: only the owner can read/write their own tickets
    match /tickets/{ticketId} {
      allow read, write: if request.auth != null
                           && request.auth.uid == resource.data.userId;
      allow create: if request.auth != null; // allow upload even before userId is
    }

    // Draw results: anyone authenticated can read (public results), only backend v
    match /results_4d/{doc} {
      allow read: if request.auth != null;
      allow write: if false; // only backend service account writes
    }
    match /results_toto/{doc} {
      allow read: if request.auth != null;
      allow write: if false;
    }
  }

  // Push tokens: only the owner can register their own device
  match /devices/{deviceId} {
    allow read, write: if request.auth != null;
  }
}

```

Click **Publish**. Without these rules, Firestore defaults to open access (anyone can read/write all data).

For local development: You can temporarily use `allow read, write: if true;` to avoid auth errors while building features — but never leave this in production.

14. Architecture Overview



Data Flow for a New Ticket Upload

1. User takes photo of ticket
2. Front end sends image to POST /api/tickets/upload
3. Backend sends image to OCR.Space → receives raw text
4. parseTicketText() extracts: gameType, drawDate, numbers, betType
5. expandSystemBet() generates all combinations (TOTO System only)
6. classifyDrawDate() decides: is the draw 'past' or 'future'?
7. If 'past': scrape official results → compare → store win/loss
8. If 'future': mark as pending → cron job will check later
9. Ticket saved to Firestore with all data
10. Full JSON response sent back to front end
11. Front end shows result card with numbers, status, prize details

15. API Reference

Base URL

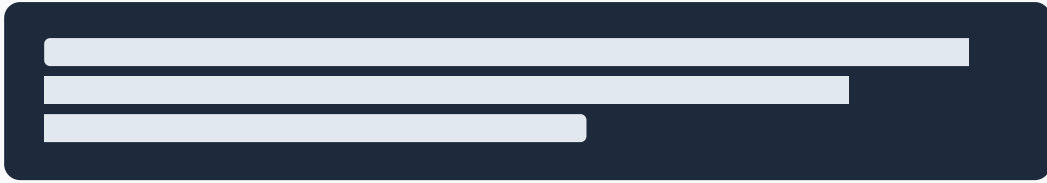
- Local: `http://localhost:3001`
- Production: Set by environment

Authentication

Most endpoints accept an optional Bearer token. Unauthenticated users can still upload and view tickets (public demo mode).

```
Authorization: Bearer <firebase-id-token>
```

How to get a Firebase ID token for testing in Postman or curl:



The Web API Key is found in Firebase Console → Project Settings → General → Your apps → Web API Key. Tokens expire after 1 hour. Regenerate when you get a 401 Unauthorized response.

POST `/api/tickets/upload`

Upload a lottery ticket image for OCR processing.

Request: `multipart/form-data`

Field	Type	Required	Description
<code>ticket</code>	File (image)	Yes	JPG, PNG, HEIC, or any image format

Response (success):

```
{
  "ticketId": "abc123",
  "gameType": "4D",
  "drawDate": "15/03/2026",
  "numbers": ["5888", "4392", "1234"],
  "betType": "Ordinary",
  "systemSize": null,
  "combinationCount": 3,
  "expandedCombinations": null,
  "serialNumber": "A12345678",
  "amount": "3.00",
  "drawType": "past",
  "resultStatus": "not_won",
  "comparison": {
    "won": false,
    "matches": [],
    "status": "checked"
  },
  "officialResult": {
    "first": "1234",
    "second": "5678",
    "third": "9012",
    "starters": ["2345", "3456", ...],
    "consolation": ["4567", "5678", ...]
  }
}
```

GET `/api/tickets`

List all tickets.

Query parameters:

Param	Type	Description
<code>gameType</code>	<code>'4D'</code> or <code>'TOTO'</code>	Filter by game type
<code>limit</code>	number	Maximum results (default: 50)

GET `/api/tickets/:id`

Get a single ticket by Firestore document ID.

DELETE `/api/tickets/:id`

Delete a ticket. Returns `{ success: true }`.

GET `/api/results/4d`

Get the latest 4D draw result.

Query parameters:

Param	Type	Description
<code>date</code>	<code>DD/MM/YYYY</code>	Specific draw date (optional)

Response:

```
{
  "drawDate": "15/03/2026",
  "drawNumber": "4522",
  "first": "1234",
  "second": "5678",
  "third": "9012",
  "starters": ["2345", "3456", "4567", "5678", "6789", "7890", "8901", "9012", "0123"],
  "consolation": ["1357", "2468", ...],
  "source": "live"
}
```

GET `/api/results/toto`

Get the latest TOTO draw result.

Response:

```
{
  "drawDate": "13/03/2026",
  "drawNumber": "3927",
  "winningNums": [3, 12, 18, 25, 34, 41],
  "addlNum": 7,
  "group1Prize": "$3,000,000",
  "source": "live"
}
```

POST `/api/results/check/:ticketId``

Manually trigger a result check for a specific ticket.

GET `/api/predict``

Run all three prediction models on both 4D and TOTO simultaneously.

Query parameters: None. The endpoint always runs all three models using the last 50 past 4D draws and last 100 past TOTO draws.

Response:

```
{
  "predictions": [
    {
      "model": "Digit Frequency Analysis",
      "modelId": "frequency",
      "why": "...",
      "predicted4D": "5832",
      "predictedTOTO": [3, 12, 18, 25, 34, 41, 7, 19, 22, 30, 38, 46],
      "confidenceScore": 0.12,
      "disclaimer": "For educational purposes only."
    },
    { "modelId": "hot_cold", "predicted4D": "...", "predictedTOTO": [...] },
    { "modelId": "odd_even", "predicted4D": "...", "predictedTOTO": [...] }
  ],
  "dataPoints": { "fourd": 50, "toto": 100 },
  "generatedAt": "2026-03-15T10:00:00.000Z",
  "disclaimer": "All predictions are purely for educational purposes and are NOT in"
}
```


POST `/api/devices/register`

Register a mobile device for push notifications.

Request body:

```
{
  "token": "ExponentPushToken[xxx]",
  "platform": "android"
}
```

16. How to Make a Change (Intern Development Loop)

This section walks you through the exact steps for making any change to this codebase — from writing the code to submitting it for review. Follow this every time, without skipping steps.

Step 1 — Create a feature branch

Never work directly on `main` or `develop`. Always create your own branch:

```
git checkout develop          # start from develop
git pull origin develop       # make sure you have the latest code
git checkout -b feature/your-feature-name # create your branch
```

Name your branch descriptively: `feature/add-4d-history-filter`, `fix/ocr-date-parsing`, `docs/update-api-reference`.

Step 2 — Make your change

Identify which layer your change belongs to (use the table in Section 2):

- Changing how something **looks**? → edit the `views/` file
- Changing what **data** is loaded or how it is processed? → edit the `presenters/` file
- Changing a **backend route or parsing logic**? → edit the relevant file in `backend/routes/` or `backend/services/`

- Changing **database structure**? → update both the backend save logic and the frontend read logic; Firestore has no migrations

While making changes:

- Keep the backend running in Terminal 1 (`node server.js`)
- Keep the web app running in Terminal 2 (`npm run dev`)
- Open a third terminal for git commands

Step 3 — Test your change manually

For a backend change, use curl to verify the endpoint still works:

```
# Example: after changing the ticket upload parser
curl -X POST http://localhost:3001/api/tickets/upload \
  -F "ticket=@/path/to/test-ticket.jpg"
```

For a web app change, open `http://localhost:5173` and manually exercise the affected flow end-to-end (not just the component you changed — the whole user journey).

For a mobile change, scan the Expo QR code and test on a real device or emulator. Do not assume the simulator is enough — always test the tab bar and safe-area insets on a real Android device.

Minimum test checklist before committing:

- ☐ Backend starts without errors
- ☐ The specific feature I changed works correctly
- ☐ The features I did NOT change still work (no regressions)
- ☐ No red errors in the browser console
- ☐ No red errors in the backend terminal

Step 3b — If you edited the manual, rebuild HTML + PDF

If your change included edits to `docs/USER_MANUAL.md`, regenerate the output files before committing:

```
node docs/build.js          # from the project root – rebuilds HTML + PDF
```

Check that the HTML opens correctly in your browser and the TOC links work. The PDF and HTML are git-ignored, so they do not need to be committed — only `USER_MANUAL.md` and `docs/build.js` go into git.

Step 4 — Commit your change

Stage only the files you intentionally changed:

```
git status                # review what changed
git add backend/routes/tickets.js # add specific files – never `git add .` blindly
git add web-app/src/views/...
git commit -m "feat: improve 4D number extraction for iBet tickets"
```

Commit message format:

```
feat:    a new feature
fix:     a bug fix
refactor: code change that doesn't add a feature or fix a bug
docs:    documentation only
test:    adding or fixing tests
```

Never commit: `serviceAccountKey.json`, `.env`, `node_modules/`, or any file containing passwords or API keys. These are in `.gitignore` — if git tries to stage them, something is wrong; investigate before proceeding.

Step 5 — Push and open a Pull Request

```
git push -u origin feature/your-feature-name
```

Then open GitHub, go to the repository, and click **"Compare & pull request"**. Set the base branch to `develop` (not `main`).

In your PR description, answer these three questions:

1. **What did I change?** (one sentence)
2. **Why?** (what problem does it solve or what was wrong before?)
3. **How did I test it?** (what exact steps did you use to verify it works?)

A reviewer will look at your PR and either approve it or leave comments. Address all comments before it is merged.

Common mistakes interns make — avoid these

Mistake	Why it's a problem	How to avoid
Pushing directly to <code>main</code>	Breaks the production-ready branch	Always branch off <code>develop</code>
Running <code>git add .</code>	Accidentally commits <code>.env</code> or <code>serviceAccountKey.json</code>	Add files by name
Editing a View to fetch data	Breaks the Presenter pattern	Re-read Section 2 layer rules
Testing only in the browser	Misses mobile-specific bugs (tab bar, safe area)	Always test on real device
Not reading the backend terminal	Misses server-side errors that the frontend silently swallows	Keep backend terminal visible
Committing <code>node_modules/</code>	Adds 100MB+ to the repo	Confirm <code>.gitignore</code> is working first

17. Intern Challenge — Improve OCR Accuracy

This section describes a **real open bug** in the codebase that an intern can attempt to fix. It is not a toy exercise — it affects real users.

Background: The Problem

When OCR.Space is unavailable (rate-limited or down), the app falls back to Tesseract.js with a custom HSV watermark removal step. This fallback **silently drops numbers** on some tickets. A confirmed real case is documented with a screenshot in [Section 11 — OCR Known Fault Case](#).

Ticket printed: A. 8537 · B. 9724 · C. 5089 · D. 9743 **App extracted:** 8537 · 9724 · 9743 ← **C. 5089 missing, no error raised**

The user uploaded a valid ticket, got a result card back, but one of their numbers was never saved. If 5089 won a prize, the app would incorrectly tell them they lost.

Why It Happens — Technical Detail

File to read first: `backend/routes/tickets.js` lines 401–453

The Tesseract fallback pipeline:

1. Upscales the image to $\geq 2400\text{px}$
2. Scans every pixel in HSV colour space — if **hue is in the red range ($< 30^\circ$ or $> 330^\circ$) AND saturation > 0.18** , the pixel is turned white
3. Converts to greyscale $\rightarrow$ normalises $\rightarrow$ runs Tesseract

The flaw is in step 2. The Singapore Pools red "S" watermark bleeds colour into nearby ink. When "5089" sits directly over a dense part of the watermark, the printed digits' pixels can also have a reddish hue and get erased. Tesseract then sees a blank line and produces no text for it.

The specific threshold that causes the problem:

```
// backend/routes/tickets.js — line 434
if ((hue < 30 || hue > 330) && sat > 0.18) {
  rawPixels[i] = rawPixels[i + 1] = rawPixels[i + 2] = 255; // white out pixel
}
```

`sat > 0.18` is too low — it erases mildly warm-toned black ink in addition to the red watermark.

Three Approaches to Investigate

Approach A — Tighten the HSV threshold (lowest effort, try this first)

The saturation threshold `0.18` is the likely culprit. Black ink printed on a white ticket typically has near-zero saturation regardless of lighting. The red watermark has saturation 0.5–0.9. Try raising the threshold:

```
// Try raising to 0.35 or higher — measure on several real ticket photos
if ((hue < 30 || hue > 330) && sat > 0.35) {
```

Test by uploading the fault-case ticket after each change and checking that all four numbers appear in the backend log's raw OCR text. A saturation threshold that removes the watermark

without touching the digit ink is the goal.

How to test: Look at the backend terminal after upload:

```
[upload] OCR raw text:
---
A. 8537 BIG $1 SML $0
B. 9724 BIG $1 SML $0
C. 5089 BIG $1 SML $0 ← this line must appear for the fix to work
D. 9743 BIG $1 SML $0
---
```

Approach B — Run Tesseract twice and merge results (medium effort)

Run the Tesseract pass twice: once with watermark removal, once without. Merge the results by taking the union of all extracted numbers. This won't eliminate false positives but will reduce false negatives (missed numbers).

The alphabet-loop parser in `parseTicketText()` already deduplicates with a `Set`, so running it on a concatenation of both raw text outputs is safe.

```
// Pseudocode – implement in backend/routes/tickets.js
const textWithRemoval = await runTesseract(processedBuffer); // existing
const textWithoutRemoval = await runTesseract(greyOnlyBuffer); // new: no HSV st
const mergedText = textWithRemoval + '\n' + textWithoutRemoval;
const extracted = parseTicketText(mergedText); // dedup handled
```

Approach C — Wire up Google Vision (highest effort, best outcome)

This is the permanent fix, blocked by the billing issue.

Current state: `backend/.env` has `GOOGLE_VISION_KEY_PATH=./google-vision-key.json`. The key file exists but the Google Cloud billing account is not activated.

Steps if billing becomes available:

1. Enable the Cloud Vision API in Google Cloud Console
2. In `backend/routes/tickets.js`, after the OCR.Space block and before Tesseract, add:

```
// Approach C scaffold – add between OCR.Space and Tesseract blocks
const vision = require('@google-cloud/vision');
const vClient = new vision.ImageAnnotatorClient({
  keyFilename: process.env.GOOGLE_VISION_KEY_PATH,
});
const [result] = await vClient.textDetection({ image: { content: ocrImageBuffer.toString('base64') } });
const visionText = result.fullTextAnnotation?.text || '';
if (visionText.length > 20) {
  text = visionText;
  ocrEngine = 'google-vision';
} else {
  throw new Error('Vision returned too little text');
}
```

3. Install the client: `cd backend && npm install @google-cloud/vision`
4. Add `google-vision-key.json` to `backend/.gitignore` if not already there

How to Measure Success

Before claiming a fix works, test it against all of these conditions:

- ❑ Ticket with 4 numbers → extracts all 4 (covers the C. 5089 fault case)
- ❑ Ticket with 1 number → extracts the 1 number
- ❑ TOTO ticket (6 numbers, 1-49) → extracts all 6
- ❑ Blurry / low-light photo → degrades gracefully (no crash, partial result is OK)
- ❑ Correct numbers only – no extra phantom numbers added by the change

Use the backend log to verify: every expected number must appear in `OCR raw text` AND in `Parsed result`.

Submitting Your Fix

Follow the dev loop in Section 16. When writing your PR description, answer:

1. Which approach did you take (A, B, or C)?
 2. What saturation threshold or logic change did you make and why?
 3. Paste the backend log output for the fault-case ticket before and after your change, showing C. 5089 now appears
-

